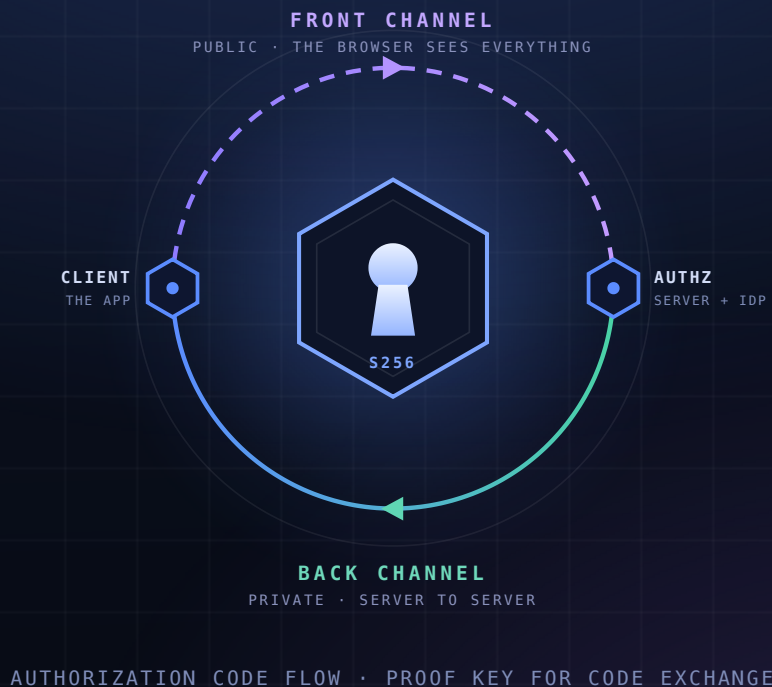


THE ULTIMATE DEVELOPER'S GUIDE TO

OAUTH 2.1 and PKCE

Delegated authorization from first principles — the flow, the cryptography, the attacks each rule prevents, and a runnable simulator to take it all apart.



10 hands-on labs · the complete source of an OAuth 2.1 server, API and client · in the book

AUTHOR

Smiroh Dev

Smiroh Publishers
FIRST EDITION

The Ultimate Developer's Guide to OAUTH 2.1 and PKCE

Smiroh Dev

PUBLISHER	Smiroh Publishers
EDITION	First edition
PUBLISHED	September 2026
FORMAT	Electronic book (PDF), 54 pages
ISBN (EB00K)	To be assigned
SUBJECT	Computers / Security / Web Development

Copyright © 2026 Smiroh Dev. All rights reserved.

Published by Smiroh Publishers. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, or other electronic or mechanical methods — without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

TRADEMARKS

All trademarks and registered trademarks appearing in this book are the property of their respective owners. They are used editorially, to the benefit of the trademark owner, with no intention of infringement. The publisher is not associated with any product or vendor mentioned in this book.

STANDARDS REFERENCED

This book discusses OAuth 2.1 and related specifications published by the Internet Engineering Task Force (IETF) and the OpenID Foundation. Those documents are the normative sources; where this book and a specification disagree, the specification governs. A complete reading list appears in the final chapter.

LIMIT OF LIABILITY AND DISCLAIMER OF WARRANTY

The information in this book is provided on an “as is” basis, without warranty of any kind, express or implied. While every precaution has been taken in its preparation, neither the author nor the publisher shall have any liability to any person or entity with respect to any loss or damage caused, or alleged to have been caused, directly or indirectly by the information contained in it. Security is a moving field: verify any practice described here against the current specifications and your own threat model before relying on it.

ABOUT THE COMPANION SIMULATOR

The software accompanying this book is a teaching tool. It deliberately implements insecure configurations so that they can be demonstrated, generates ephemeral keys, keeps all state in memory, and serves plain HTTP on `localhost`. It must never be deployed, exposed to a network, or used to protect anything real. Code listings are illustrative; production systems should use maintained, independently audited OAuth libraries and authorization servers.

What is in this guide

Read it in order the first time. Each chapter assumes only the ones before it.

01	The problem OAuth solves	Why handing over your password is the wrong answer	4
02	The four roles	Resource owner, client, authorization server, resource server	5
03	Front channel and back channel	The one distinction the whole protocol is built on	6
04	The Authorization Code flow	All eleven steps, drawn and then narrated	7
05	The attack PKCE prevents	What a stolen authorization code used to be worth	11
06	PKCE, precisely	Three lines of arithmetic, and why they are enough	12
07	Anatomy of a token	Inside a JWT; access vs ID vs refresh tokens	14
08	Scopes and consent	How authority is narrowed, three times over	16
09	Refresh tokens and rotation	Long sessions from short-lived tokens	17
10	What OAuth 2.1 changed	Mostly a list of deletions, and that is the point	18
11	How an API validates a token	Four checks, in order, on every request	19
12	The mistakes that matter	A checklist drawn from real incidents	20
13	Run it yourself	Every concept here, as a runnable lab	21

Part II — The Labs

–	Setup, and how to run it	22	A	crypto-lab.js	27
1–3	The flow, and the proof	23	B	authz-server.js	30
4–6	Breaking the rules	24	C	api-server.js	41
7–8	Scope, and token confusion	25	D	client.js	45
9–10	Machine tokens; revocation	26		1,363 lines, complete	
–	Glossary	Every term in one place			53
–	The specifications	Which RFC says what, and which to read first			54

A note on terminology. OAuth is about **authorization** (what an app may do), not **authentication** (who you are). The two get muddled constantly because in practice they arrive together. Chapter 7 separates them: authentication is what OpenID Connect adds on top.

The problem OAuth solves

An application needs data that belongs to you, held by somebody else. There is an obvious way to arrange that, and it is a catastrophe.

Suppose a budgeting app wants to read your bank transactions. The naive design is to ask you for your banking username and password and log in as you. Every single thing wrong with that idea turns out to be something OAuth fixes.

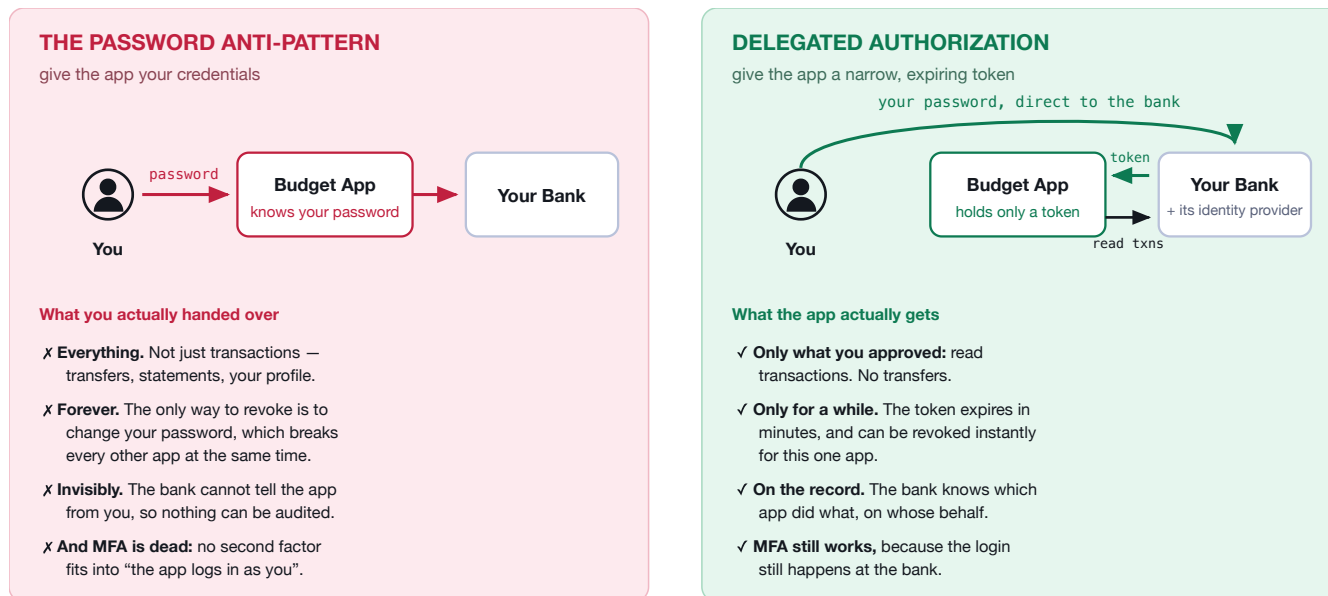


Figure 1.1 — The same goal, two arrangements. On the right, the password travels only between you and the party that issued it; the application is handed a credential that is narrow by construction.

The one-sentence definition

OAuth 2.1 is a protocol for letting an application act on your behalf, in a limited way, without ever seeing your password. You authenticate to the party that already knows you; that party issues the application a scoped, expiring, revocable token.

The rest of this guide is about doing that *safely* when the exchange must pass through a web browser — the single constraint, explored in Chapter 3, that makes OAuth look harder than the sentence above.

Three things OAuth is not

Not authentication

An access token says *what may be done*, not *who is here*. Treating one as proof of identity is a vulnerability class of its own; identity is a separate token (Chapter 7).

Not encryption

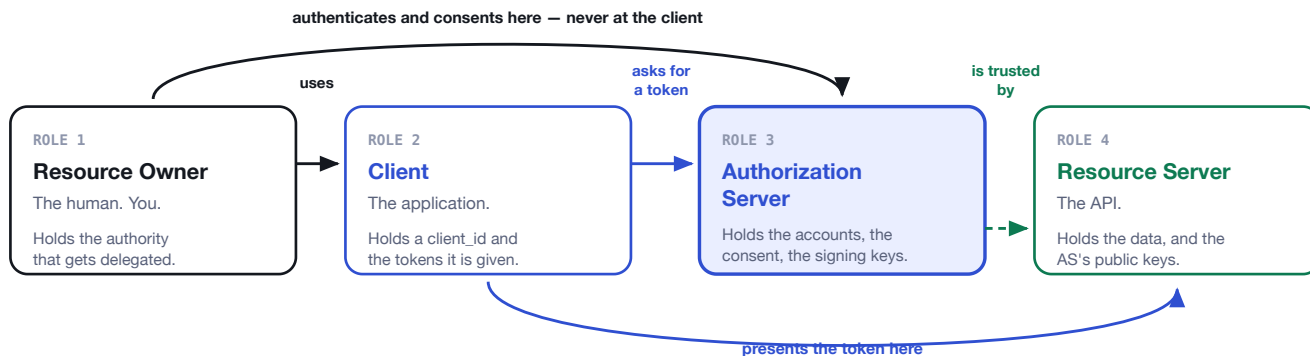
A bearer token is a password-equivalent in plain text: anyone who copies it can use it. OAuth assumes **TLS everywhere**.

Not a permission system

Scopes are coarse labels on an app's authority. Whether *this user* may touch *this record* stays your API's decision, on every request.

The four roles

OAuth defines four participants. Almost every confusing sentence in the specifications becomes clear once you know which of the four is being talked about.



The asymmetry that makes it work:

the **client** never learns who you are, and the **API** never learns how you proved it. Each knows only what it needs.

Figure 2.1 — The four roles and the relationships between them. Roles 3 and 4 are often the same company but never the same code path: issuing tokens and accepting them are different jobs.

The same four roles, three ways

ROLE	BANK EXAMPLE	“SIGN IN WITH GOOGLE”	IN THE SIMULATOR
Resource Owner	You, the account holder	You, the Google user	alice or bob
Client	The budgeting app	The site you are signing in to	:9020
Authorization Server	The bank's login service	Google's OAuth service	:9000
Resource Server	The bank's accounts API	Gmail / Drive / Calendar APIs	:9010

Two kinds of client, and why it matters so much

This distinction decides how a client may authenticate, and it is why PKCE exists.

Confidential client HAS A SECRET

Code that runs on a server you control. It can keep a `client_secret` because nobody can read its memory or disk.

Examples: a server-rendered web app, a backend service.
Proves identity by: sending its secret to the token endpoint.

Public client CANNOT KEEP A SECRET

Code that runs on someone else's device. Anyone can read the JavaScript bundle or decompile the binary, so any embedded secret is published to the world.

Examples: a single-page app, a mobile app, a CLI. **Proves identity by:** PKCE, and nothing else.

The mistake to avoid. A single-page app is a **public** client however private your bundle feels: a secret shipped to a browser or an app store is not a secret. OAuth 2.1's answer is to stop depending on one — PKCE is mandatory for every client, so correctness no longer rests on this classification.

Front channel and back channel

If you remember one idea from this guide, make it this one. Every rule in OAuth 2.1 — including all of PKCE — follows from the difference between these two ways of moving data.

The client and the authorization server need to talk. They have two completely different means of doing so, with completely different security properties.

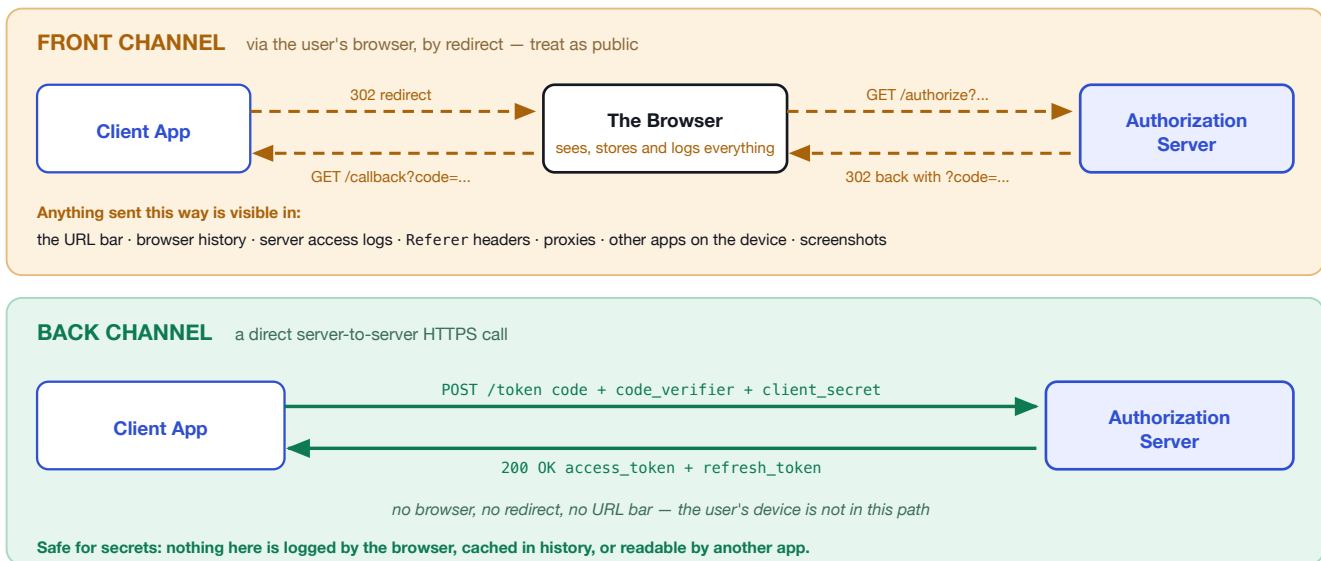


Figure 3.1 — The same two parties, two channels. Dashed arrows are browser redirects; solid arrows are direct HTTPS calls.

The rule that follows

May cross the front channel

- `client_id` — public by definition
- `redirect_uri`, `scope`, `state`
- `code_challenge` — a hash, not a secret
- the `code` — useless on its own (*this is the whole trick*)

Must never cross it

- `client_secret`
- `code_verifier`
- `access_token`, `refresh_token`, `id_token`
- the user's password, obviously

This is why the implicit grant was removed. It returned an *access token* on the front channel, in the URL fragment — landing it in browser history and in any Referrer header the page emitted. Chapter 10 has the full list of what OAuth 2.1 deleted and why.

“Front channel” does not mean unencrypted

Both channels run over TLS in production. The front channel is dangerous not because it is exposed in transit, but because of where the data **comes to rest**: browsers write URLs into history, servers into access logs, pages leak them through `Referer`. TLS does nothing about any of that. OAuth 2.1 still requires `https` everywhere, with two carve-outs — `http://localhost` for development, and private-use schemes like `com.example.app:/callback` for native apps. That is why the simulator may run on plain HTTP, and why nothing else should.

The Authorization Code flow

All of OAuth 2.1 in one picture; the next two pages walk it step by step.

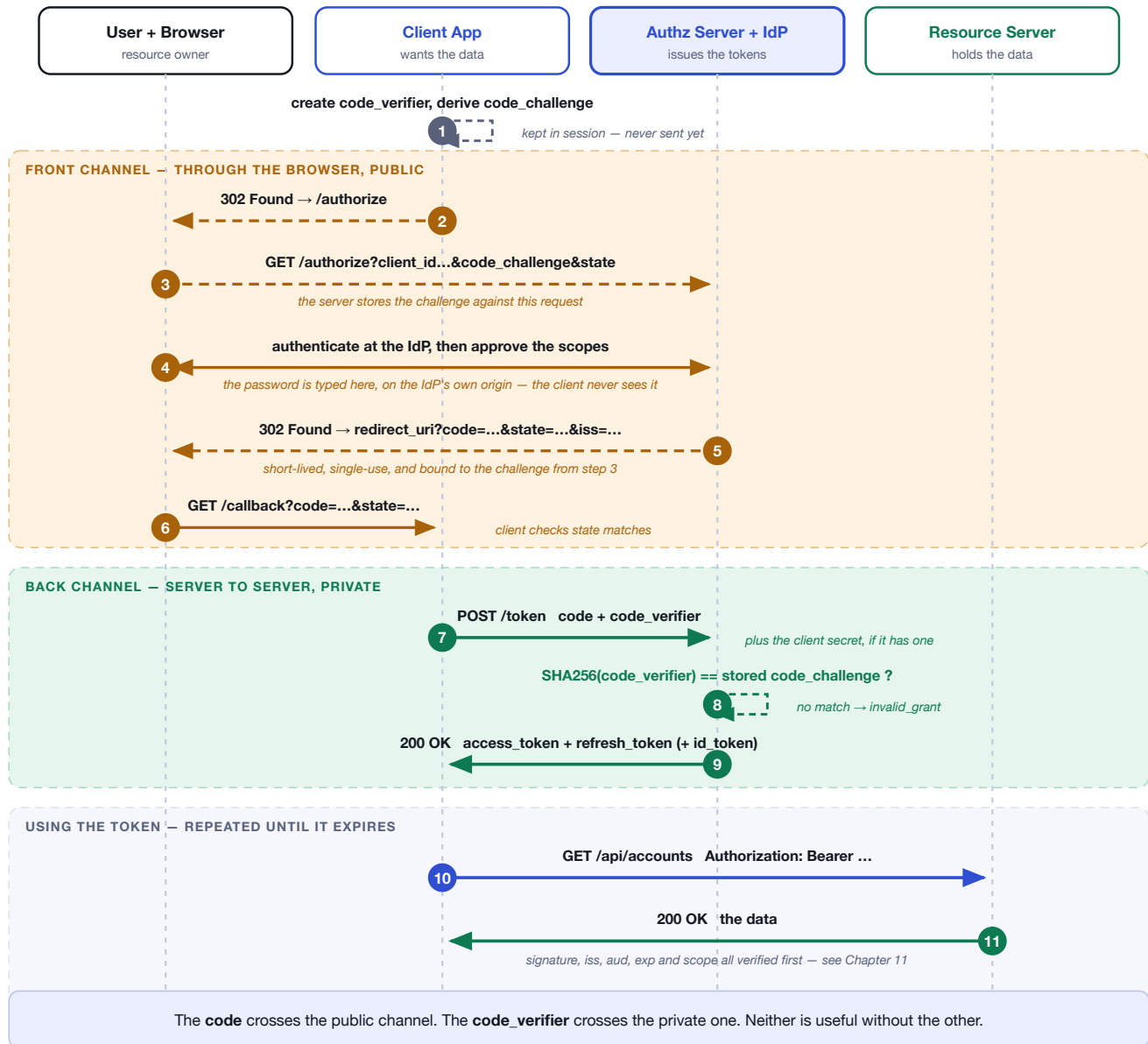


Figure 4.1 — Dashed arrows are browser redirects, solid arrows are direct HTTP calls, looped arrows are local computation. Steps 10–11 repeat until the token expires.

Three things to notice

The user is a participant

Steps 2–6 happen in a browser because a human has to be asked. That is not overhead — it is the consent.

The code is a claim ticket

It is not a token and grants nothing: a single-use reference, valid about a minute, redeemable only by whoever started the flow.

Two channels, one flow

The split at step 7 is the design: the risky public hop carries something worthless, the valuable exchange happens out of reach.

Step by step: the front channel

The same flow again, this time as real HTTP. Every value below was taken from an actual run of the simulator.

1 · The client invents a secret it will not send

Before anything touches the network, the client generates a high-entropy random string and hashes it. The random string is the **code_verifier**; the hash is the **code_challenge**.

```
# 32 random bytes, base64url-encoded => exactly 43 characters
code_verifier = 3TbF_QUTB0iHthQd0rRr_6GcCTQKN_nPqc9j3P0FCBg

# BASE64URL(SHA256(ASCII(code_verifier)))
code_challenge = kPX0H7VuZIrt1ZXIk7n0C2HkA6r0YNI3lDys2XF1Crk
```

The verifier is stored in the client's own session. It must survive until step 7 and must never appear in a URL, a cookie readable by script, or a log line.

2–3 · Redirect the user to the authorization endpoint

The client does not call `/authorize` itself. It sends a `302` so that the *browser* goes there, carrying the parameters in the query string.

```
GET /authorize?response_type=code
    &client_id=demo-web-app
    &redirect_uri=http%3A%2F%2Flocalhost%3A9020%2Fcallback
    &scope=openid+profile+accounts%3Aread
    &state=cclt66TixPmn7BXxpsyNEA
    &code_challenge=kPX0H7VuZIrt1ZXIk7n0C2HkA6r0YNI3lDys2XF1Crk
    &code_challenge_method=S256

Host: localhost:9000
```

PARAMETER	WHAT IT IS FOR
response_type	Always <code>code</code> in OAuth 2.1. The other values are gone.
client_id	Which application is asking. Public, not a credential.
redirect_uri	Where to send the code. Compared by exact string match against registration.
scope	What the application wants to be allowed to do. Ask for the least you need.
state	Random, per-request. Echoed back so the client can recognise its own callback.
code_challenge	The hash from step 1. The server stores it against this request.
code_challenge_method	S256 . Send it explicitly — omitting it means <code>plain</code> , which protects nothing.

Step by step: the exchange

The user takes over, and hands back a code that is worth nothing by itself. Turning it into a token takes a direct call, and a proof.

4–6 • The user authenticates, consents, and comes back with a code

Everything in this step happens on the authorization server's own origin. The client is not running, cannot see the password, and cannot influence the consent screen. When it is done, the server redirects the browser back:

```
HTTP/1.1 302 Found
Location: http://localhost:9020/callback?code=bnBKJfGg0H4Mm_fzI4bE622-5t0w7U8kbKIHpubbQ4o
&state=cclt66TixPmn7BXxpsyNEA
&iss=http://localhost:9000
```

Two checks the client must make here. First, state has to equal the value it generated in step 2 — otherwise this callback was not started by this browser session, and the code must be dropped. Second, iss must name the authorization server the flow began with (RFC 9207), which prevents a code from one server being replayed at another.

7 • Redeem the code, and prove you started the flow

This is a server-to-server `POST`. The browser is not involved and never sees any of it.

```
POST /token HTTP/1.1
Host: localhost:9000
Content-Type: application/x-www-form-urlencoded
Authorization: Basic ZGVtbY13ZWItYXBwOndlYi1hcHAtd3VwZXItc2VjcmV0  <- only if confidential

grant_type=authorization_code
&code=bnBKJfGg0H4Mm_fzI4bE622-5t0w7U8kbKIHpubbQ4o
&redirect_uri=http://localhost:9020/callback
&code_verifier=3TbF_QUTB0iHthQd0rRr_6GcCTQKN_nPqc9j3P0FCBg
```

8 • What the authorization server checks, in order

1. Does this `code` exist, and has it been used before? A replay means someone has a copy; every token it produced is revoked.
2. Has it expired? Codes live about a minute, because they cross the browser.
3. Was it issued to *this* `client_id`? A code belongs to one client.
4. Does `redirect_uri` match the one from step 3? Both halves of the flow must agree.
5. Does `SHA256(code_verifier)` equal the stored `code_challenge`?
6. If the client is confidential, did it authenticate correctly?

Any failure returns `400 invalid_grant` and no token — and check 5 is the only one an attacker holding a stolen code cannot satisfy.

Step by step: the tokens

What comes back, how it is used, and what the failures mean.

9 · The tokens come back

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store

{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJodHRwOi8v...",
  "token_type": "Bearer",
  "expires_in": 300,
  "scope": "openid profile accounts:read",
  "refresh_token": "hz9ZLvVJ2akqMFGb7pQx...",
  "id_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJodHRwOi8v..."
}
```

`Cache-Control: no-store` is required, not decorative: these are credentials. Note too that `scope` comes back in the response — the user may have granted **less** than was asked for, and a correct client reads what it received.

10–11 · Call the API

```
GET /api/accounts HTTP/1.1
Host: localhost:9010
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
```

The token goes in the `Authorization` header — never in a query string, where it lands in every access log on the path. OAuth 2.1 removes the query-string option OAuth 2.0 allowed.

When it goes wrong: decoding the error

ERROR	USUAL CAUSE
invalid_grant	The code was used already, expired, issued to another client, or the <code>code_verifier</code> did not match. Deliberately vague: the server will not tell an attacker which check failed.
invalid_client	Wrong <code>client_id</code> or secret, or a confidential client that sent no credentials at all.
invalid_request	A required parameter is missing or duplicated. Commonly a forgotten <code>code_challenge_method</code> .
unauthorized_client invalid_scope	The client is not registered for this grant type, or asked for a scope it is not allowed (or that does not exist).
access_denied	The user pressed Deny. Not a bug; handle it gracefully.

That is the entire flow. Eleven steps, two channels, one hash comparison. Every remaining chapter is either a detail of what is inside the tokens, or a rule protecting one of these steps.

The attack PKCE prevents

The authorization code has to cross the browser, and the browser is not a private place. So assume the code leaks — and ask what it is worth.

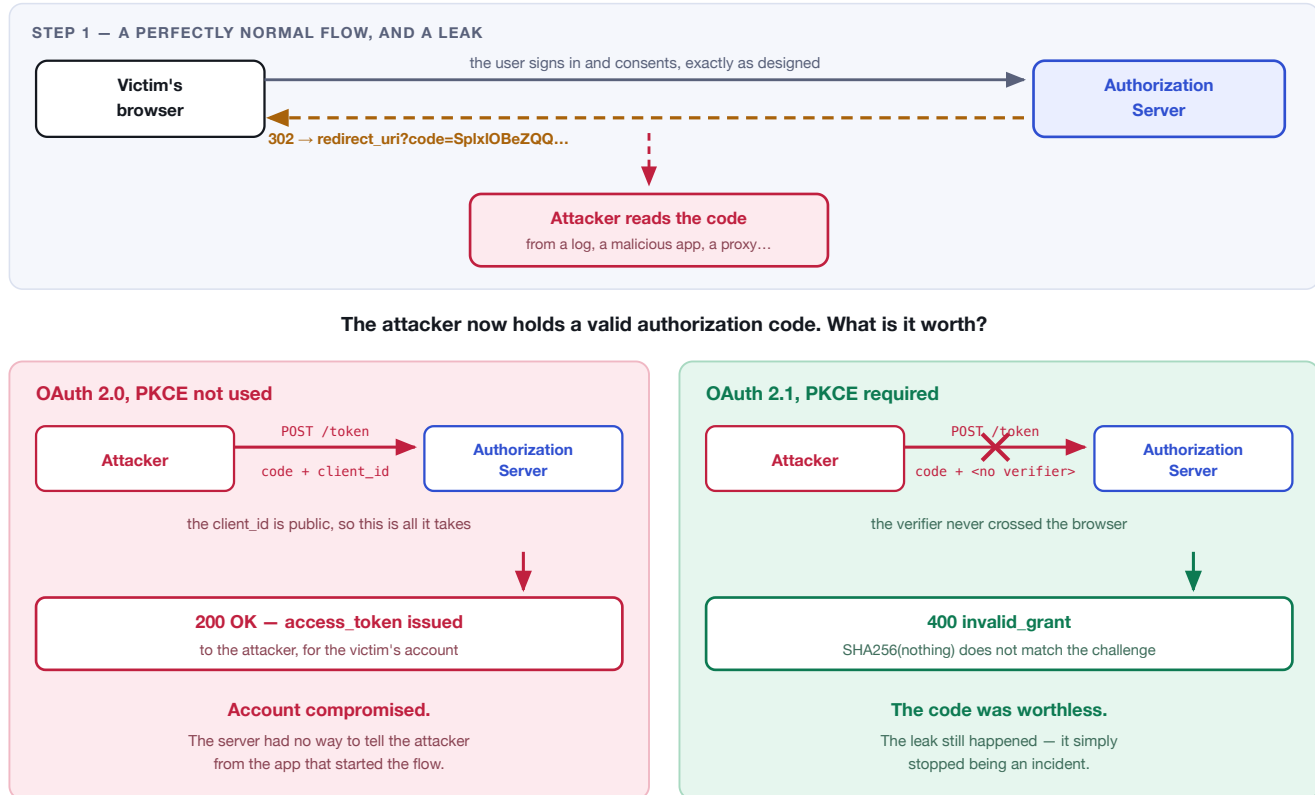


Figure 5.1 — Identical theft, two outcomes. PKCE does not prevent the code from leaking; it removes the value of having it.

How codes actually leak

None of these are exotic. Every one has produced real CVEs:

- **A malicious app on the same device** registering the same custom URI scheme (`myapp://callback`) — the OS may hand the redirect to either app.
- **Server access logs.** The callback URL, complete with code, is written to disk by every proxy and web server in the path.
- **The `Referer` header** leaking the callback URL to any third-party script or image on the landing page.
- **An open redirect** on the client's own domain, used to bounce the code to the attacker's server.
- **Browser history** on a shared or managed machine.
- **A TLS-terminating proxy** on a corporate or hostile network.

Why OAuth 2.1 requires PKCE even for confidential clients. A server-side app with a secret is already protected against most of this — the attacker would need the secret too. But PKCE also protects against a code being *injected* into an honest client's session, costs nothing, and removes any need to reason about which category a client falls into. Universal rules fail safe; conditional ones get misapplied.

PKCE, precisely

Proof Key for Code Exchange (RFC 7636). One random string, one hash, one comparison — and the authorization code stops being worth stealing.

The name is unhelpful; the idea is simple. Before starting the flow the client invents a secret, sends only a *fingerprint* of it through the browser, and later proves it holds the original. Only the app that began the flow can finish it.

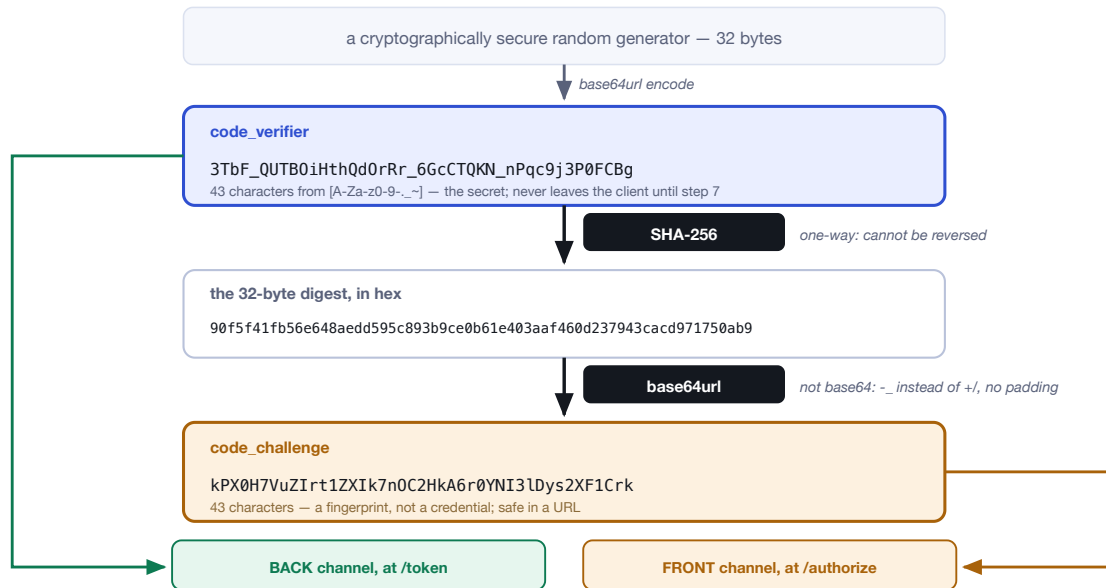


Figure 6.1 — Deriving the challenge from the verifier, with real values. The two results travel on different channels, and only one of them is a secret.

Why a hash is enough

The attacker sees the challenge

It is right there in the authorization URL. To use the stolen code they would need a string that hashes to it — that is a SHA-256 preimage, which nobody can compute. The challenge is public *by design*.

The attacker cannot guess the verifier

It is 256 bits of randomness. There is no dictionary, no pattern and no structure to attack. This is why the specification insists on a *cryptographically secure* generator, not `Math.random()`.

Never use `code_challenge_method=plain`. With `plain`, the “challenge” is the verifier, sent in the clear through the browser — so anyone who sees the authorization request can redeem the code. It exists in RFC 7636 only for devices that genuinely could not compute SHA-256 in 2015. Worse, RFC 7636 says that an **omitted** `code_challenge_method` defaults to `plain`. Always send S256 explicitly.

The check, and how to write it

What the server does with the two halves, and the code that produces them.

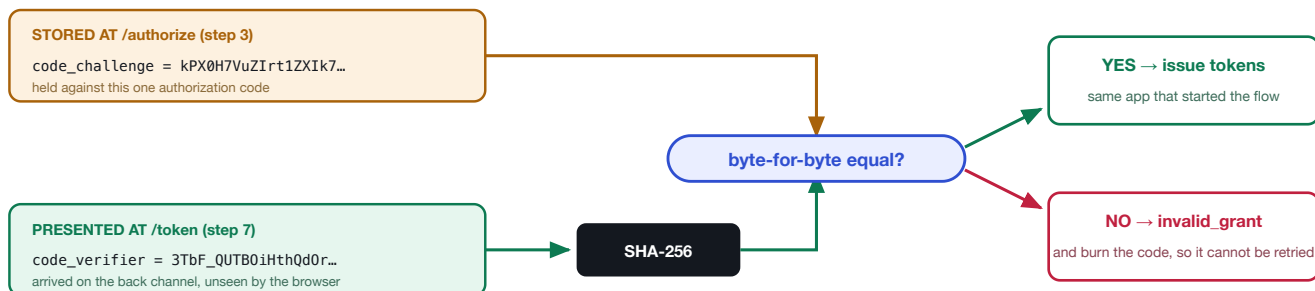


Figure 6.2 — The whole verification. Compare in **constant time**, so a rejection cannot be timed byte by byte, and treat a failure as a security event, not a retry.

Generating the pair

Browser (Web Crypto)

```

const b64url = (b) => btoa(
  String.fromCharCode(...b))
  .replace(/\+/g, '-')
  .replace(/\//g, '_')
  .replace(/=+$/, '');

const verifier = b64url(
  crypto.getRandomValues(new
    Uint8Array(32)));

const hash = await crypto.subtle.digest(
  'SHA-256',
  new TextEncoder().encode(verifier));

const challenge = b64url(new
  Uint8Array(hash));
  
```

Node.js

```

const crypto = require('node:crypto');

const verifier = crypto.randomBytes(32)
  .toString('base64url');

const challenge =
  crypto.createHash('sha256')
    .update(verifier).digest('base64url');
  
```

Python

```

import base64, hashlib, os

b64 = lambda b:
  base64.urlsafe_b64encode(b).rstrip(b'=')

verifier = b64(os.urandom(32))
challenge =
  b64(hashlib.sha256(verifier).digest())
  
```

In all three, the SHA-256 is taken over the **base64url text** of the verifier, never over the raw random bytes. Getting that backwards is the most common PKCE bug and the hardest to spot, because both values still look like valid base64url.

One pair per authorization request. A verifier is a nonce, not a configuration value: generate a fresh one every time and discard it the moment the code is exchanged. On the server side, compare with `timingSafeEqual` rather than `==`, and treat a mismatch as a security event — invalidate the code, return the generic `invalid_grant`, and never reveal which check failed. The common implementation mistakes are collected in Chapter 12.

Anatomy of a token

An access token is usually a JWT: three base64url segments joined by dots. Two of them are plain text that anyone can read.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50L3NpdCI6ImtpZCI6InNpbS1FYXJIZkN0.eyJpc3MiOiJodHRwOi8vbG9jYXRob3N0OjkwMDAiLCJzZW50L3NpdCI6ImtpZCI6InNpbS1FYXJIZkN0.OkNChhb8y_TlJlTQquscjcfwn2VKkzJZPB0fzVrsgjNsesPiNaCJaZZgq5QxL
```

1 · HEADER

Which algorithm signed it, and which key (kid) to verify with.

Readable by anyone. Never trust its aLg blindly.

2 · PAYLOAD (CLAIMS)

Who, what, where, until when. This is the authorization itself.

Also readable by anyone — so never put a secret in here.

3 · SIGNATURE

RSA-SHA256 over segments 1+2, made with the AS's private key.

The only part that cannot be forged or edited.

Figure 7.1 — A JWT is not encrypted. Base64url is an encoding, not a cipher: paste any token into a decoder and read it. The signature does not hide the contents — it proves nobody changed them.

What the segments say once decoded

```
// segment 1 - the header
{ "alg": "RS256", "typ": "at+jwt", "kid": "sim-EarHfCN6MZTq0Jjo" }

// segment 2 - the claims
{ "iss": "http://localhost:9000", // who issued it
  "sub": "u-1001", // whose data it acts on
  "aud": "http://localhost:9010", // which API may accept it
  "client_id": "demo-web-app", // which app is holding it
  "scope": "openid profile accounts:read", // what it may do there
  "jti": "78e3f1b14a7c70e11fac6fc1", // unique id, for revocation
  "iat": 1789228222, "nbf": 1789228222, // issued at / not before
  "exp": 1789228522 } // expires - 300 seconds later
```

Why typ: at+jwt matters

RFC 9068 gives access tokens their own type, so an API can reject anything that is not one — an ID token, for instance. Without it, two different tokens from the same issuer look interchangeable.

Opaque tokens are also fine

Plenty of servers issue random strings instead. The API then calls `/introspect` to find out what a token means. Same protocol, different trade-off — see Chapter 11.

A token is a bearer credential. Whoever holds it can use it, exactly as with a password. There is no cryptographic binding to the client that received it, which is why they are short-lived, scoped and audience-restricted — three ways of limiting the damage rather than preventing the theft.

Three tokens, three jobs

A single token response can contain three completely different things. Confusing two of them is the most common OAuth bug in production code.

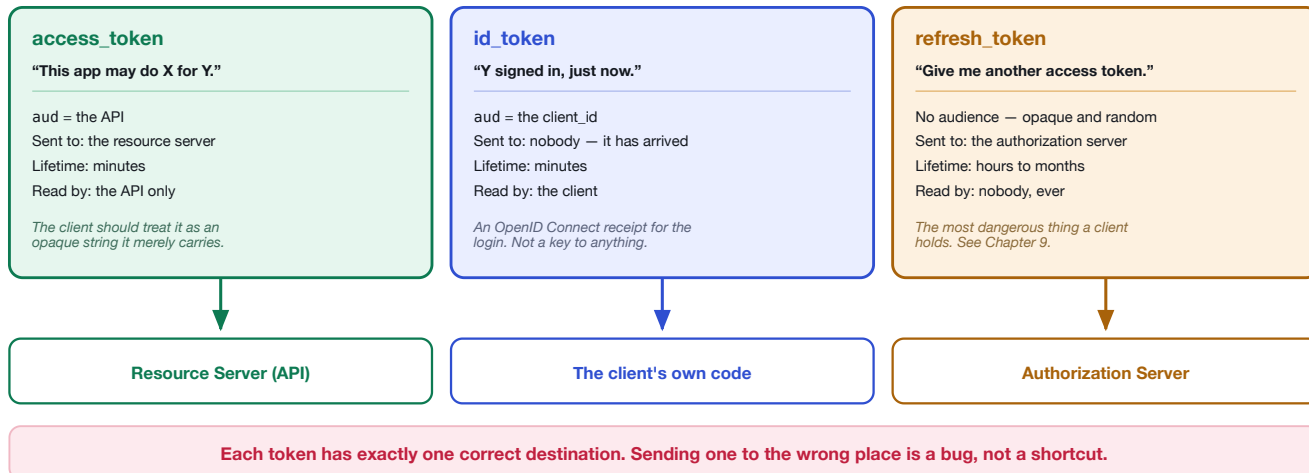


Figure 7.2 — Three tokens, three consumers. The aud claim is not documentation: it is the mechanism that enforces this separation.

The mistake, and why the API must refuse it

An ID token is a JWT from the same issuer, so it is tempting to send it as a Bearer token. It looks valid. It is not:

```
# The ID token's own claims say who it is for:
"aud": "demo-web-app"           # the CLIENT, not the API
"typ": "JWT"                    # not at+jwt
# ...and it carries no `scope` at all, so there is nothing to authorize.
GET /api/accounts Authorization: Bearer <id_token> -> 401 invalid_token
```

An API accepting an ID token is accepting a credential minted for somebody else, with no statement of what it may do — and in a federated setup that is exploitable.

The habit to build. When a call fails with 401, check which token you sent before you check anything else. “I sent the ID token” accounts for a remarkable share of lost afternoons. Lab 08 in the simulator does exactly this, twice.

Where OpenID Connect fits

OAuth alone never tells the client *who* the user is; it only hands over authority. OpenID Connect is a thin layer that adds exactly that. “Sign in with Google” is OpenID Connect, “let this app read your Drive” is plain OAuth, and most deployments do both at once in the one flow from Chapter 4:

- Ask for the `openid` scope, and the token response gains an `id_token`.
- Send a random `nonce`; it comes back inside the ID token, proving the token was minted for *this* login rather than replayed.
- A `/userinfo` endpoint returns profile claims, filtered by granted scope.

Scopes and consent

A scope is a label for a slice of authority. What an application asks for and what it gets are two different things, and it is narrowed three separate times.

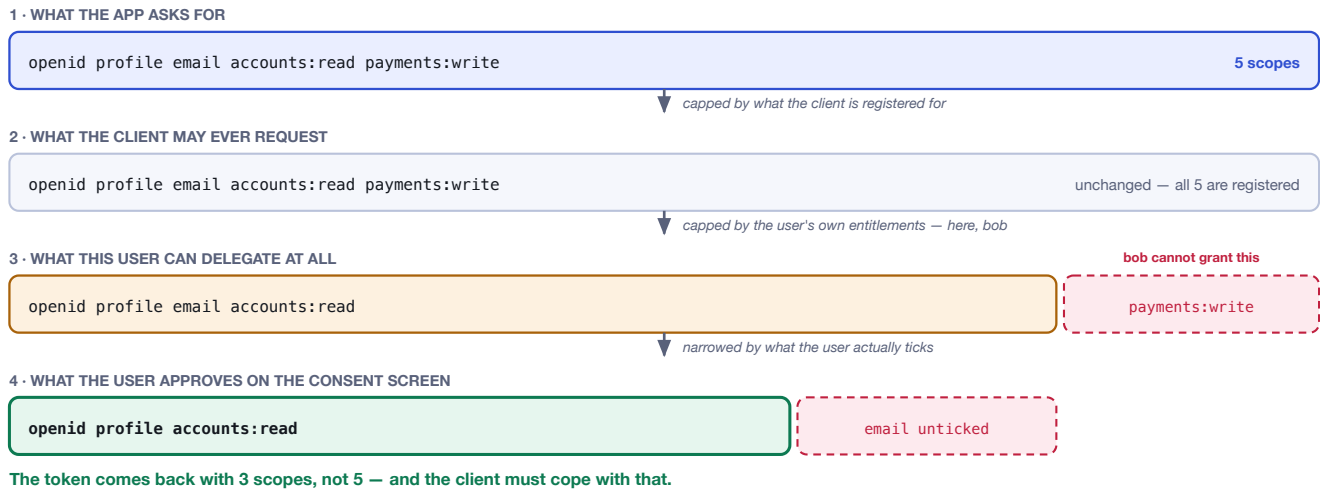


Figure 8.1 — Scope only ever narrows. Each stage can remove, never add, which is why the token response repeats the scope it actually granted.

Designing scopes people can consent to

Good

- `accounts:read`, `payments:write` — resource plus verb
- Separate read from write, always
- Few enough to fit on one screen
- Named for what the *user* loses if abused

Bad

- `admin`, `full_access` — unconsentable
- One scope per API endpoint — unreadable
- Scopes encoding record ids (`account:4410`)
- Internal role names leaked into the UI

Scope is not the same as permission

This distinction causes real vulnerabilities. A scope says *this application may perform this kind of operation on this user's behalf*. It says nothing about which records that user may touch.

```
# The token carries scope=accounts:read and sub=u-1001.
GET /api/accounts/ACC-7100      # <- an account belonging to u-1002

# A scope check alone would ALLOW this. The API must also check ownership.
```

The simulator's API takes the user from the token's `sub` and never from the request, which is the habit worth copying. An object reference supplied by the caller must always be authorized against the token's subject.

Refresh tokens and rotation

Access tokens expire in minutes. Users expect to stay signed in for weeks. The refresh token bridges that gap — and needs careful handling because of it.

An access token cannot be recalled once issued, so it is given a short life: a leak is a problem for five minutes, not five months. The refresh token mints replacements without troubling the user.

ONE TOKEN FAMILY, OVER TIME

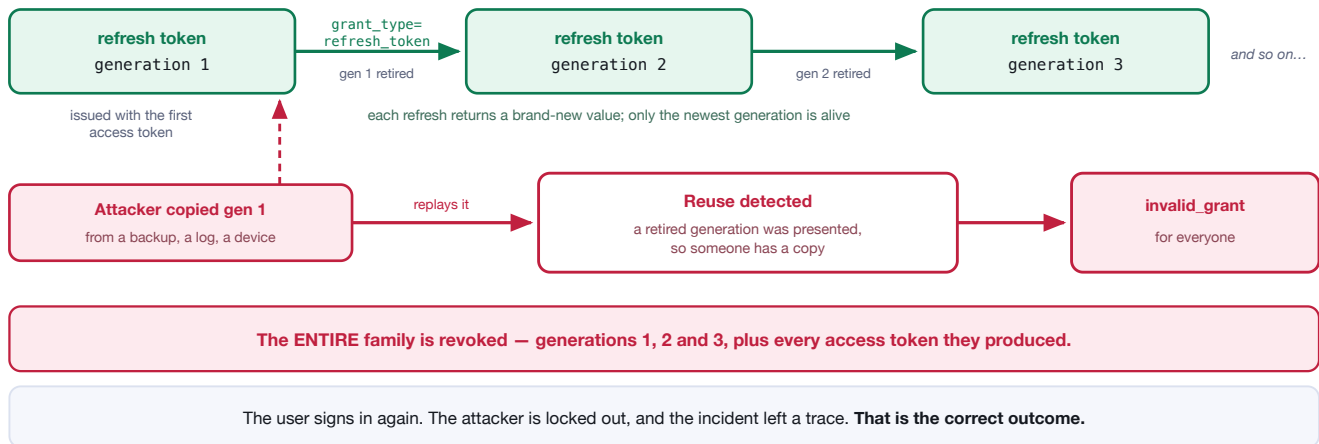


Figure 9.1 — Rotation with reuse detection (RFC 9700 §4.14). The server cannot tell thief from victim, so it trusts neither: a silent, permanent compromise becomes a visible, recoverable one.

What this buys you

- **A stolen refresh token stops working.** Whichever party refreshes second is refused, and the theft is detected rather than lasting for months.
- **The theft becomes observable.** A reuse event is a high-quality security signal, because correct clients never replay a retired token. This is why OAuth 2.1 requires rotation (or sender-constraining) for public clients.

How long should each one live?

CREDENTIAL	TYPICAL	WHY
authorization code	30–60 s	It crosses the browser and is redeemed immediately. Anything longer is pure exposure.
access token	5–15 min	Cannot be recalled, so its lifetime is the exposure window.
refresh token	hours–months	Long, but rotating and revocable. Shorter for public clients.
IdP session	hours	Decides whether a <i>new</i> authorization needs a fresh login.

The operational catch. If a client crashes after sending a refresh but before storing the new token, it will replay the old one and get the whole family revoked — logging the user out for no good reason. Persist the rotated token before you use the access token that came with it, and serialise refreshes so two tabs cannot race.

What OAuth 2.1 changed

OAuth 2.1 invents almost nothing. It deletes what proved dangerous and makes the best existing practice mandatory — and that is precisely why it is an improvement.

OAuth 2.0 is a framework of options. A decade of deployments showed which options were traps, and the security Best Current Practice (RFC 9700) documented them. OAuth 2.1 consolidates that guidance into the core, so that the default reading of the specification is now the safe one.

OAUTH 2.0 (2012)	OAUTH 2.1	WHY IT CHANGED
PKCE optional, recommended for mobile	Required for every authorization code request	Codes leak in ways nobody predicted, the defence costs almost nothing, and a universal rule cannot be misapplied.
Implicit grant (<code>response_type=token</code>)	Removed	It put an access token in the URL fragment: browser history, <code>Referer</code> headers, any script on the page — and the client could not be authenticated at all.
Password grant (<code>grant_type=password</code>)	Removed	The application handled the real password, so there was no consent, no MFA, no federation, and no way for a user to tell a real app from a phishing one.
<code>redirect_uri</code> matching left to the server	Exact string comparison	Wildcards and prefix rules let an attacker append a path and have codes delivered to themselves.
Bearer tokens allowed in query strings	Header only	Query strings are written to every access log, proxy log and browser history on the path.
Refresh tokens static and long-lived	Rotate, or bind to the client	A stolen static refresh token is permanent, undetectable access. Rotation makes the theft visible and recoverable.
Little guidance on mixing up servers	iss in the authorization response	Prevents a mix-up attack where a code from a malicious server is replayed at an honest one.

If you already know OAuth 2.0

You know OAuth 2.1. Add PKCE everywhere, stop using implicit and password, match redirect URIs exactly, and rotate refresh tokens. That is the entire migration.

On the specification's status

OAuth 2.1 is an IETF draft that consolidates published, stable RFCs. Nothing in it is speculative — every requirement already exists as a standard somewhere. Implementing it is not a bet on a draft.

The pattern worth noticing. Every deletion removes a *choice* a developer could get wrong. The most reliable security improvement is not a new mechanism but the removal of an unsafe option — which is the actual lesson of OAuth 2.1.

How an API validates a token

The resource server is the part most tutorials skip. On every single request it asks four questions, in this order.

AN INCOMING REQUEST

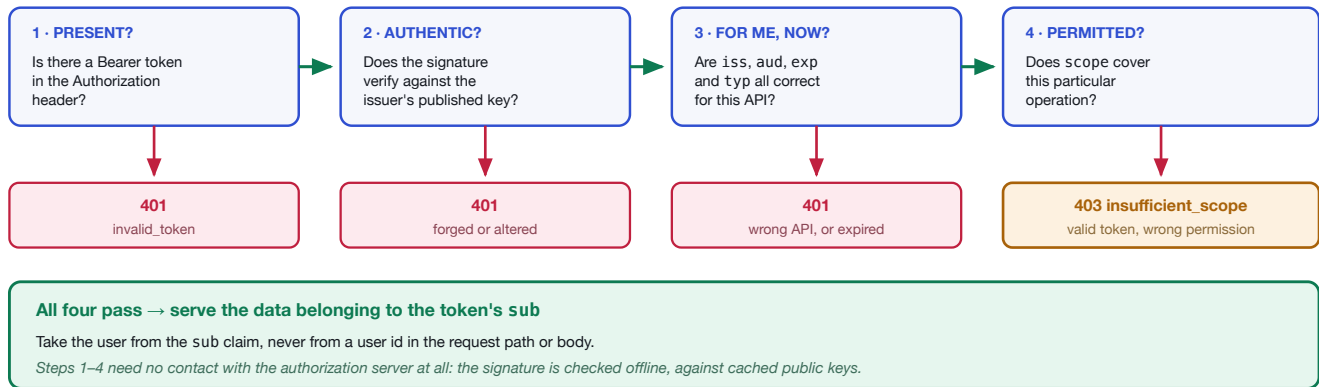


Figure 11.1 — The four gates. Note the status codes: **401** means “this token is not acceptable”; **403** means “the token is fine but does not authorize this”. Returning 401 for a scope failure sends clients into pointless refresh loops.

Local validation or introspection?

Verify the JWT locally

Fetch the issuer's public keys once from `/jwks.json`, cache them by `kid`, verify offline.

Fast and needs no network. **But** it cannot see a revocation: a revoked token stays acceptable until `exp`.

Ask the authorization server

Call `/introspect` (RFC 7662) per request. Works with opaque tokens too.

Always current, including revocation. **But** it adds a round trip to every call and makes the AS a hard dependency.

This trade-off is the real reason access tokens are short-lived: a five-minute window is the compromise between not calling the AS on every request and not honouring a revocation for too long. Lab 12 in the simulator revokes a token and then calls both kinds of endpoint, so you can watch one accept it and the other refuse.

Three ways to get validation wrong

- **Trusting the token's own `alg` header.** If you let the token choose its algorithm, an attacker sends `alg: none` (no signature at all) or swaps RS256 for HS256 and signs with your *public* key as the HMAC secret. Pin the expected algorithm in your code and reject everything else.
- **Skipping the `aud` check.** Then any token from that issuer works at your API, including one minted for a low-value service that has no business here. A valid signature only says who issued a token, not who it was for.
- **Caching keys without handling rotation.** Cache by `kid`, refetch when an unknown `kid` appears, and rate-limit that refetch. An issuer that rotates keys should not take your API down — nor should an unknown `kid` let an attacker trigger unbounded fetches.

The mistakes that matter

A checklist drawn from the failures that actually recur.

Writing a client

MISTAKE	CONSEQUENCE
Hashing the raw random bytes instead of the base64url verifier text	Every exchange fails with <code>invalid_grant</code> and nothing in the error says why.
Using base64 instead of base64url	<code>+</code> , <code>/</code> and <code>=</code> get percent-encoded in the URL, and the comparison fails.
Reusing one <code>code_verifier</code> for every request	A single leak compromises every future flow. It is a nonce: generate a fresh pair each time.
Keeping the verifier or tokens in <code>localStorage</code>	Readable by any script on the origin, so one XSS takes everything. Keep them server-side, or in memory.
Omitting <code>code_challenge_method</code>	RFC 7636 defaults it to <code>plain</code> : a strict server rejects you, a lenient one silently drops your protection.
Assuming the granted scope equals the requested scope	Features break confusingly when a user declines one checkbox. Read <code>scope</code> from the response.
Sending the <code>id_token</code> to the API	<code>401</code> , and an afternoon lost. It is a login receipt, not an API key.

Running a server or an API

MISTAKE	CONSEQUENCE
Accepting the token's own <code>alg</code> header	Opens <code>alg: none</code> and RS256→HS256 confusion. Pin the algorithm server-side.
Not validating <code>aud</code>	A token minted for any client of that issuer works at your API.
Prefix or wildcard <code>redirect_uri</code> matching	An attacker appends a path and has your users' codes delivered to them.
Letting an authorization code be redeemed twice	Whoever holds a copy can keep minting tokens. Single-use, and revoke on replay.
Returning <code>401</code> for a scope failure	Clients refresh in a loop trying to fix something a refresh cannot fix. Use <code>403</code> .
Taking the user id from the request instead of <code>sub</code>	A textbook IDOR sitting on top of otherwise correct OAuth.

A useful instinct. When OAuth surprises you, ask *which channel is this on, and who can see it?* Nearly every mistake above is a value that ended up on the wrong channel, or a check skipped because the happy path worked without it.

Run it yourself

Reading about a protocol gets you halfway. Part II of this book is a complete OAuth 2.1 deployment in four files — an authorization server, an API and a client — with ten labs that make each chapter happen on your own machine.

Everything you need is in this book

The four listings in Part II are the whole program: 1,363 lines, no elisions, no repository to clone, and **no dependencies to install**. They run on Node's own `node:http` and `node:crypto`, which is the point — a library would hide exactly the parts this book is about.

```
# three terminals; the first two narrate what the protocol is doing
node authz-server.js      # port 7000 - the AS and IdP
node api-server.js        # port 7010 - the protected API
node client.js happy      # Lab 1 - the correct flow, end to end
```

Each chapter, as a lab

LAB	COMMAND	WHAT IT PROVES	CHAPTER
1	<code>client.js happy</code>	The whole flow, then a call the token is not scoped for	3–4
2	<code>client.js pkce</code>	The verifier, the hash, the challenge, and the comparison	6
3	<code>client.js attack</code>	A stolen code is worthless without the verifier	5–6
4	<code>client.js nopkce</code>	PKCE off: the same theft succeeds	5–6
5	<code>client.js replay</code>	Codes are single-use, and a replay revokes what it made	4
6	<code>client.js refresh</code>	Rotation, then reuse detection killing the family	9
7	<code>client.js scope</code>	Scope narrowing, and <code>403 insufficient_scope</code>	8
8	<code>client.js idtoken</code>	ID-token and wrong-audience tokens both refused	7, 11
9	<code>client.js m2m</code>	<code>client_credentials</code> : no user, no consent	2
10	<code>client.js revoke</code>	Local validation still accepts a revoked token	11

The command worth running twice. Lab 3 steals an authorization code and is refused. Lab 4 sets `require_pkce` to `false` on the running server and steals the same way — and walks off with a working access token for the victim's account. Identical theft, opposite outcomes, one flag apart. Every safety rule in `authz-server.js` can be switched like that: `curl localhost:7000/policy` lists them.

The Labs

Everything from here on is runnable. The four listings in this part are complete — nothing is elided, nothing is downloaded. Type or copy them into four files and you have a working OAuth 2.1 authorization server, a protected API, and a client that drives ten experiments against them.

What you need

Node.js 18 or newer, and nothing else. There is no `package.json`, no `npm install` and no dependency to audit: every line runs on Node's own `node:http` and `node:crypto`. That is deliberate. A dependency would hide exactly the parts of the protocol this book is about.

The four files

FILE	LINES	LISTING	WHAT IT IS
<code>crypto-lab.js</code>	126	A	PKCE and JWT primitives. Required by the other three.
<code>authz-server.js</code>	591	B	The authorization server and identity provider, on port 7000.
<code>api-server.js</code>	196	C	The resource server, on port 7010.
<code>client.js</code>	450	D	The client, and all ten labs.

Put all four in one directory — the three servers do `require('./crypto-lab')`, so they expect it alongside them.

Starting it up

Three terminals. The first two stay running and print what the protocol is doing; the third is where you work.

```
# terminal 1 – the authorization server and IdP
node authz-server.js

# terminal 2 – the API
node api-server.js

# terminal 3 – the labs
node client.js list
node client.js happy
```

The client needs no browser. It walks the front channel with a cookie jar, following redirects and filling in the login and consent forms itself, so each lab is one command whose result is printed rather than clicked. You can still open `http://localhost:7000/authorize?...` by hand if you want to see the screens a real user would.

Keep terminal 1 in view. The authorization server narrates every decision it makes — which rule it checked, what it compared, and why the request passed or failed. Reading a chapter here and then watching the matching line appear there is the fastest way to make any of this stick.

Labs 1–3: the flow, and the proof

Start here. The first three labs build the correct flow, take the PKCE transform apart, and then try to steal from it.

LAB 01

The correct flow, end to end

```
node client.js happy
```

Every step of Chapter 4 in order: the PKCE pair, the authorization request, login, consent, the code, the token exchange, the decoded access token, then a call to the API and a call the token is *not* scoped for.

PASS state matches – this callback is ours

PASS tokens issued

PASS 200 – 2 accounts

PASS 403 insufficient_scope – the API enforced scope

Try this: change `scope` in the call to `getCode()` to just `'accounts:read'`. The response has no `id_token` any more, because you stopped asking for `openid` — that one scope is the whole of OpenID Connect.

LAB 02

The PKCE transform, step by step

```
node client.js pkce
```

Prints a verifier, its SHA-256 in hex and base64, and the resulting challenge, so you can watch base64 become base64url. Then it verifies a correct verifier and a different one.

PASS correct verifier -> true

PASS a different one -> false (pkce_mismatch)

Try this: in `crypto-lab.js`, change `update(verifier, 'ascii')` to hash the raw random bytes instead of the encoded text. Lab 1 now fails with `invalid_grant` and nothing tells you why — the most common PKCE bug, reproduced in one edit.

LAB 03

A stolen authorization code, refused by PKCE

```
node client.js attack
```

Takes a legitimately issued code and hands it to an attacker, who tries to redeem it with no verifier and then with an invented one. Uses the **public** client, so there is no client secret quietly doing the work.

PASS REFUSED: invalid_grant

PASS REFUSED: the hash of a guess never matches

Why it matters: the leak still happened. PKCE does not prevent the code from escaping — it removes the value of having it.

Labs 4–6: breaking the rules

Each of these turns off a requirement of OAuth 2.1 and shows you what the requirement was for. Lab 4 is the one to run twice.

LAB 04

The same theft, with PKCE turned off

```
node client.js nopkce
```

Sets `require_pkce` to `false` on the running server, obtains a code with no `code_challenge` at all, and lets the attacker redeem it. It restores the safe default when it finishes.

FAIL ATTACK SUCCEEDED – the attacker holds a real access token

FAIL and it WORKS: `{"id":"ACC-4410","balance":4820.55}`

Run Lab 3 and Lab 4 back to back. Identical theft, opposite outcomes, one policy flag apart. Nothing else in this book makes the case for PKCE as quickly.

LAB 05

Replaying an already-used code

```
node client.js replay
```

Redeems one code twice. The second attempt is refused, and every token the first redemption produced is revoked — RFC 9700's answer to a code that may be in two places at once.

PASS tokens issued

PASS REFUSED: codes are single-use

Try this: `curl -X POST localhost:7000/policy -d 'key=single_use_codes&value=false'`, then run it again and watch the replay mint a second set of tokens.

LAB 06

Refresh rotation and reuse detection

```
node client.js refresh
```

Refreshes once and shows that a *different* refresh token comes back, then replays the retired one as a thief would. The whole family is revoked, including the copy the honest client was still holding.

PASS ROTATED – a brand-new refresh token came back

PASS REFUSED and the whole family revoked

PASS also refused – the server cannot tell thief from victim

Why it matters: the user gets logged out. That is not the bug, that is the feature — a silent permanent compromise became a loud recoverable one.

Labs 7–8: scope, and what a token is for

Two labs on the questions an API has to answer after the signature checks out.

LAB 07

Scope is capped three times over

```
node client.js scope
```

Signs in as **bob**, who is not entitled to `payments:write`, and asks for it anyway. The consent screen cannot offer what the user does not hold, so the token comes back narrower than requested — then the API refuses the operation.

```
requested openid profile accounts:read payments:write
actually granted openid profile accounts:read
PASS payments:write was dropped – the user never had it to give
PASS 403 insufficient_scope
```

Try this: run it again as `alice`, who *is* entitled. The payment now succeeds with `201`. Same client, same code, same request — different resource owner.

LAB 08

An ID token is not an access token

```
node client.js idtoken
```

Compares the two tokens one flow returns, then commits the classic mistake: sending the ID token to the API as a Bearer token. Finally it uses the `resource` parameter (RFC 8707) to mint a genuine, unexpired token for a *different* API and presents that instead.

```
access typ / aud at+jwt / http://localhost:7010
id typ / aud JWT / demo-web-app
PASS 401 unexpected_typ – refused, as it must be
PASS 401 audience_mismatch – refused
```

Why it matters: the second token has a perfect signature from the right issuer and has not expired. The only thing wrong with it is `aud` — which is precisely what confines a token to one API.

Labs 9–10: no user, and life after revocation

The last two labs cover the grant with no human in it, and the trade-off that explains every short token lifetime you have ever seen.

LAB 09

Machine to machine: no user, no consent

```
node client.js m2m
```

Gets a token with `client_credentials` — no browser, no redirect, no consent screen — then asks the API who the user is.

```
sub demo-service
refresh_token none (correct)
PASS 404 no_such_subject - correct, nobody delegated
```

Why it matters: `sub` is the client id. There is no human behind this token and nothing was delegated, which is why it must never be used to act on a user's behalf. No refresh token either: the client can simply ask again with its own credentials.

LAB 10

Revocation: local validation vs introspection

```
node client.js revoke
```

Revokes an access token, then calls two endpoints that serve the same data — one validating the JWT locally, one asking the authorization server.

```
local validation 200 <- STILL ACCEPTED
introspection 401 <- correctly refused
```

Why it matters: a self-contained JWT knows nothing about having been revoked; its signature and `exp` are unchanged. Introspection sees the truth at the cost of a round trip per call. **That** is why access tokens live for minutes: the lifetime is the compromise between the two.

Where to go from here

Every policy flag in `authz-server.js` can be switched at runtime — `curl localhost:7000/policy` lists them. Turn one off, re-run the labs, and read the warnings the server prints. Then try the exercises: add a `/userinfo` endpoint, enforce `max_purchase`-style limits, or implement DPoP (RFC 9449) so a stolen token is useless without the client's key.

PKCE and JWT primitives

Everything cryptographic the labs need, on Node core modules. Read `verifyPkce` and `verifyJwt` and you have read the two checks the rest of the protocol rests on.

LINES 1–42 OF 126

```

1  'use strict';
2  // =====
3  // LAB FILE 1 of 4 - crypto-lab.js
4  // PKCE and JWT primitives, on Node core modules only. No dependencies.
5  // =====
6  const crypto = require('node:crypto');
7
8  // --- base64url: base64 with +/ -> _- and the = padding removed -----
9  const b64u = (buf) => Buffer.from(buf).toString('base64url');
10 const unb64u = (str) => Buffer.from(str, 'base64url');
11
12 // --- PKCE (RFC 7636) -----
13
14 // The verifier is the secret: 32 random bytes, base64url -> 43 characters.
15 const createVerifier = () => crypto.randomBytes(32).toString('base64url');
16
17 // challenge = BASE64URL(SHA256(ASCII(verifier))) <- hash the TEXT, not bytes
18 const challengeS256 = (verifier) =>
19   crypto.createHash('sha256').update(verifier, 'ascii').digest('base64url');
20
21 // Show every step of the transform, for Lab 2.
22 function explainPkce(verifier) {
23   const digest = crypto.createHash('sha256').update(verifier, 'ascii').digest();
24   return {
25     code_verifier: verifier,
26     length: verifier.length,
27     sha256_hex: digest.toString('hex'),
28     sha256_base64: digest.toString('base64'), // note the trailing '='
29     code_challenge: digest.toString('base64url'), // the same, url-safe
30   };
31 }
32
33 // The authorization server's check. Constant-time: a comparison that returns
34 // early on the first differing byte leaks the stored challenge.
35 function verifyPkce(verifier, challenge, method) {
36   if (!verifier) return { ok: false, reason: 'missing_code_verifier' };
37   if (!/^[A-Za-z0-9\-\._~]{43,128}$/.test(verifier))
38     return { ok: false, reason: 'malformed_code_verifier' };
39   const computed = method === 'plain' ? verifier : challengeS256(verifier);
40   const a = Buffer.from(computed), b = Buffer.from(challenge || '');
41   const ok = a.length === b.length && crypto.timingSafeEqual(a, b);
42   return ok ? { ok: true, computed } : { ok: false, reason: 'pkce_mismatch', computed };

```

PKCE and JWT primitives

LINES 43–84 OF 126

```

43 }
44
45 // --- Keys and JWTs -----
46
47 // The key id is a SHA-256 over the key's own material (RFC 7638 thumbprint),
48 // so new key material always announces itself with a new kid. That is what
49 // lets the API cache public keys safely. Restarting this server rotates keys.
50 function generateKey() {
51   const { publicKey, privateKey } = crypto.generateKeyPairSync('rsa', {
52     modulusLength: 2048,
53     publicKeyEncoding: { type: 'spki', format: 'pem' },
54     privateKeyEncoding: { type: 'pkcs8', format: 'pem' },
55   });
56   const jwk = crypto.createPublicKey(publicKey).export({ format: 'jwk' });
57   const thumb = crypto.createHash('sha256')
58     .update(JSON.stringify({ e: jwk.e, kty: jwk.kty, n: jwk.n }))
59     .digest('base64url');
60   const kid = 'lab-' + thumb.slice(0, 16);
61   return { kid, privateKey, publicJwk: { ...jwk, alg: 'RS256', use: 'sig', kid } };
62 }
63
64 // A JWT is three base64url segments joined by dots: header.payload.signature.
65 function signJwt(payload, key, typ) {
66   const header = { alg: 'RS256', typ: typ || 'JWT', kid: key.kid };
67   const input = b64u(JSON.stringify(header)) + '.' + b64u(JSON.stringify(payload));
68   const sig = crypto.sign('RSA-SHA256', Buffer.from(input), key.privateKey);
69   return input + '.' + sig.toString('base64url');
70 }
71
72 // Decode WITHOUT verifying. For display only – never trust this output.
73 function decodeJwt(token) {
74   const p = String(token || '').split('.');
75   if (p.length !== 3) return null;
76   try {
77     return {
78       header: JSON.parse(unb64u(p[0]).toString('utf8')),
79       payload: JSON.parse(unb64u(p[1]).toString('utf8')),
80       signingInput: p[0] + '.' + p[1],
81       signature: p[2],
82     };
83   } catch { return null; }
84 }

```

PKCE and JWT primitives

LINES 85–126 OF 126

```

85
86 // Verify the way a resource server must: signature first, then every claim
87 // that scopes the token to THIS api at THIS moment. See Chapter 11.
88 function verifyJwt(token, { getKey, issuer, audience, requiredTyp, skew = 5 }) {
89   const d = decodeJwt(token);
90   if (!d) return { valid: false, reason: 'malformed_token' };
91   // Never let the token choose its own algorithm: that is how alg:none and
92   // RS256→HS256 confusion attacks start.
93   if (d.header.alg !== 'RS256') return { valid: false, reason: 'unsupported_alg' };
94   if (requiredTyp && d.header.typ !== requiredTyp)
95     return { valid: false, reason: 'unexpected_typ' };
96   const key = getKey(d.header.kid);
97   if (!key) return { valid: false, reason: 'unknown_kid' };
98   const ok = crypto.verify('RSA-SHA256', Buffer.from(d.signingInput),
99                           key, unb64u(d.signature));
100   if (!ok) return { valid: false, reason: 'bad_signature' };
101
102   const now = Math.floor(Date.now() / 1000);
103   const p = d.payload;
104   if (issuer && p.iss !== issuer) return { valid: false, reason: 'issuer_mismatch' };
105   if (audience && ![].concat(p.aud || []).includes(audience))
106     return { valid: false, reason: 'audience_mismatch' };
107   if (typeof p.exp === 'number' && now > p.exp + skew)
108     return { valid: false, reason: 'token_expired' };
109   return { valid: true, payload: p, header: d.header };
110 }
111
112 // --- odds and ends -----
113 const randomToken = (n = 32) => crypto.randomBytes(n).toString('base64url');
114 const randomId = (n = 8) => crypto.randomBytes(n).toString('hex');
115 const sha256hex = (v) => crypto.createHash('sha256').update(v).digest('hex');
116
117 function safeEqual(a, b) {
118   const x = Buffer.from(String(a ?? '')), y = Buffer.from(String(b ?? ''));
119   return x.length === y.length && crypto.timingSafeEqual(x, y);
120 }
121
122 module.exports = {
123   b64u, unb64u, createVerifier, challengeS256, explainPkce, verifyPkce,
124   generateKey, signJwt, decodeJwt, verifyJwt,
125   randomToken, randomId, sha256hex, safeEqual,
126 };

```

The authorization server and IdP

The whole of Chapter 4 as running code: `/authorize` with its six validations, login, consent, and a `/token` endpoint handling three grants. The `policy` object near the top is what Lab 4 switches off.

LINES 1–53 OF 591

```

1  'use strict';
2  // =====
3  // LAB FILE 2 of 4 – authz-server.js      run:  node authz-server.js
4  // A complete OAuth 2.1 authorization server + identity provider.
5  // Implements: RFC 6749 (as narrowed by 2.1), 7636 PKCE, 8414 discovery,
6  //             7662 introspection, 7009 revocation, 9068 at+jwt, 9700 BCP.
7  // =====
8  const http = require('node:http');
9  const C = require('./crypto-lab');
10
11  const PORT = 7000;
12  const ISSUER = 'http://localhost:7000'; // browser-facing identity; goes in `iss`
13  const API = 'http://localhost:7010';   // the audience we mint tokens for
14
15  // --- who can log in -----
16  const USERS = {
17    alice: { sub: 'u-1001', password: 'wonderland', name: 'Alice Liddell',
18            email: 'alice@lab.test',
19            can: ['openid', 'profile', 'email', 'accounts:read', 'payments:write'] },
20    // bob deliberately cannot delegate payments:write – see Lab 6.
21    bob: { sub: 'u-1002', password: 'builder', name: 'Bob Builder',
22          email: 'bob@lab.test',
23          can: ['openid', 'profile', 'email', 'accounts:read'] },
24  };
25  const userBySub = (sub) => Object.values(USERS).find((u) => u.sub === sub);
26
27  // --- which apps are registered -----
28  const CLIENTS = {
29    // A server-side app CAN keep a secret, so it authenticates to /token.
30    'demo-web-app': { secret: 'web-app-secret', type: 'confidential',
31                    redirect_uris: ['http://localhost:7020/callback'],
32                    grants: ['authorization_code', 'refresh_token'],
33                    scopes: ['openid', 'profile', 'email', 'accounts:read', 'payments:write'] },
34    // A browser or mobile app CANNOT. PKCE is its only proof of identity.
35    'demo-spa': { secret: null, type: 'public',
36                redirect_uris: ['http://localhost:7020/spa-callback'],
37                grants: ['authorization_code', 'refresh_token'],
38                scopes: ['openid', 'profile', 'accounts:read'] },
39    // No user involved at all: the app acts as itself.
40    'demo-service': { secret: 'service-secret', type: 'confidential',
41                    redirect_uris: [], grants: ['client_credentials'],
42                    scopes: ['accounts:read', 'profile'] },
43  };
44
45  const SCOPES = ['openid', 'profile', 'email', 'accounts:read', 'payments:write'];
46
47  // --- the safety rules. Every one is ON in a correct OAuth 2.1 server. -----
48  // Lab 3 turns require_pkce off and watches a stolen code start working.
49  const policy = {
50    require_pkce: true,           // PKCE for every authorization_code request
51    exact_redirect_uri: true,     // string equality, no wildcards, no prefixes
52    single_use_codes: true,       // a replayed code is revoked, not honoured
53    rotate_refresh: true,         // a new refresh token on every refresh

```

The authorization server and IdP

LINES 54–107 OF 591

```

54   detect_reuse: true,           // a replayed refresh token kills the whole family
55   };
56
57   const TTL = { code: 60, access: 300, refresh: 3600, session: 1800 };
58
59   // --- server-side state (a real AS keeps this in a database) -----
60   const sessions = new Map();    // browser is logged in to the IdP
61   const requests = new Map();    // an /authorize call parked during login
62   const codes = new Map();       // one-time authorization codes
63   const refresh = new Map();     // sha256(token) -> record, grouped into families
64   const revoked = new Set();     // access-token ids killed before exp
65
66   const KEY = C.generateKey();
67   const now = () => Math.floor(Date.now() / 1000);
68   const splitScope = (s) => String(s || '').trim().split(/\s+/).filter(Boolean);
69
70   // --- tiny http helpers -----
71   function readBody(req) {
72     return new Promise((resolve) => {
73       let raw = '';
74       req.on('data', (c) => { raw += c; if (raw.length > 1e5) req.destroy(); });
75       req.on('end', () => resolve(new URLSearchParams(raw)));
76     });
77   }
78   const cookies = (req) => Object.fromEntries((req.headers.cookie || '')
79     .split(';').map((p) => p.trim().split('=')).filter((p) => p[0]));
80
81   function json(res, status, obj) {
82     res.writeHead(status, { 'content-type': 'application/json',
83       'cache-control': 'no-store' }); // these are credentials
84     res.end(JSON.stringify(obj, null, 2));
85   }
86   const html = (res, body, status = 200) =>
87     (res.writeHead(status, { 'content-type': 'text/html; charset=utf-8' }),
88     res.end('<!doctype html><meta charset=utf-8>' +
89       '<body style="font:15px/1.5 system-ui;max-width:34em;margin:3em auto">' +
90       body));
91   const redirect = (res, location) => (res.writeHead(302, { location }), res.end());
92   const esc = (s) => String(s).replace(/[\<>"]/g, (c) =>
93     ({ '<': '&lt;', '>': '&gt;', '&': '&amp;', '"': '&quot;' }[c]));
94
95   // Deliver an error to the client by redirect. Only ever called AFTER the
96   // client_id and redirect_uri are both validated – otherwise this endpoint
97   // would be an open redirector.
98   function redirectError(res, uri, error, description, state) {
99     const u = new URL(uri);
100    u.searchParams.set('error', error);
101    if (description) u.searchParams.set('error_description', description);
102    if (state) u.searchParams.set('state', state);
103    log('reject', error + ': ' + description);
104    return redirect(res, u.toString());
105  }
106  const log = (tag, msg) => console.log(' [as] ' + tag.padEnd(7) + msg);
107

```

The authorization server and IdP

LINES 108–161 OF 591

```

108 // --- client authentication -----
109 function authenticateClient(req, body) {
110   const auth = req.headers.authorization || '';
111   let id, secret;
112   if (auth.toLowerCase().startsWith('basic ')) {
113     const raw = Buffer.from(auth.slice(6), 'base64').toString('utf8');
114     const i = raw.indexOf(':');
115     id = decodeURIComponent(raw.slice(0, i));
116     secret = decodeURIComponent(raw.slice(i + 1));
117   } else { id = body.get('client_id'); secret = body.get('client_secret'); }
118
119   const client = CLIENTS[id];
120   if (!client) return { error: 'invalid_client', description: 'Unknown client_id.' };
121   if (client.type === 'confidential') {
122     if (!C.safeEqual(secret, client.secret))
123       return { error: 'invalid_client', description: 'Client authentication failed.' };
124     log('ok', 'client authenticated: ' + id);
125   } else {
126     // A public client is IDENTIFIED, never authenticated. PKCE carries the proof.
127     log('info', 'public client identified: ' + id + ' (PKCE carries the proof)');
128   }
129   return { id, client };
130 }
131
132 // --- token minting -----
133 function issueAccessToken(clientId, sub, scope, familyId, audience) {
134   const iat = now(), jti = C.randomId(12);
135   const token = C.signJwt({
136     iss: ISSUER, sub, aud: audience || API, client_id: clientId,
137     scope: scope.join(' '), jti, iat, nbf: iat, exp: iat + TTL.access,
138   }, KEY, 'at+jwt'); // RFC 9068: an API can now reject anything that is not one
139   return { token, jti, familyId };
140 }
141
142 function issueRefreshToken(clientId, sub, scope, familyId, generation) {
143   const token = C.randomToken(32);
144   // Stored hashed, like a password: a database leak then yields nothing usable.
145   refresh.set(C.sha256hex(token), { clientId, sub, scope, familyId, generation,
146     used: false, revoked: false, expiresAt: now() + TTL.refresh });
147   return token;
148 }
149
150 function revokeFamily(familyId) {
151   let n = 0;
152   for (const r of refresh.values())
153     if (r.familyId === familyId && !r.revoked) { r.revoked = true; n++; }
154   return n;
155 }
156
157 // =====
158 // The server
159 // =====
160 const server = http.createServer(async (req, res) => {
161   const url = new URL(req.url, ISSUER);

```

The authorization server and IdP

LINES 162–215 OF 591

```

162  const q = url.searchParams;
163  const path = url.pathname;
164  const body = req.method === 'POST' ? await readBody(req) : new URLSearchParams();
165
166  // ----- discovery + keys (RFC 8414) -----
167  if (path === '/.well-known/oauth-authorization-server') return json(res, 200, {
168    issuer: ISSUER,
169    authorization_endpoint: ISSUER + '/authorize',
170    token_endpoint: ISSUER + '/token',
171    jwks_uri: ISSUER + '/jwks.json',
172    introspection_endpoint: ISSUER + '/introspect',
173    revocation_endpoint: ISSUER + '/revoke',
174    scopes_supported: SCOPES,
175    // OAuth 2.1: `code` only. The implicit grant is gone, so no response type
176    // can return a token in the front channel.
177    response_types_supported: ['code'],
178    grant_types_supported: ['authorization_code', 'refresh_token',
179                          'client_credentials'],
180    code_challenge_methods_supported: ['S256'], // never offer `plain`
181    token_endpoint_auth_methods_supported: ['client_secret_basic', 'none'],
182  });
183
184  if (path === '/jwks.json') return json(res, 200, { keys: [KEY.publicJwk] });
185  if (path === '/healthz')
186    return json(res, 200, { ok: true, issuer: ISSUER, kid: KEY.kid });
187
188  // ----- the lab switchboard -----
189  if (path === '/policy' && req.method === 'GET') return json(res, 200, policy);
190  if (path === '/policy' && req.method === 'POST') {
191    const key = body.get('key'), value = body.get('value') === 'true';
192    if (!(key in policy)) return json(res, 400, { error: 'unknown_policy_key' });
193    policy[key] = value;
194    log('policy', key + ' = ' + value + (value ? '' : '    << now running UNSAFELY'));
195    return json(res, 200, policy);
196  }
197
198  // ----- /authorize : the FRONT CHANNEL -----
199  // Runs in the browser. Everything here is visible in the URL bar, history,
200  // logs and Referer headers, so nothing secret may travel on it.
201  if (path === '/authorize') {
202    const client = CLIENTS[q.get('client_id')];
203    // Step 1: identify the client. Not yet trusted – just recognised.
204    if (!client) return html(res,
205      '<h2>invalid_client</h2><p>Unknown client_id. The authorization server will '
206      + 'not redirect an error for an unknown client, because it has no registered '
207      + 'URI it can trust.</p>', 400);
208
209    // Step 2: validate redirect_uri BEFORE trusting it.
210    const uri = q.get('redirect_uri') || '';
211    const match = policy.exact_redirect_uri
212      ? client.redirect_uris.includes(uri)
213      : client.redirect_uris.some((r) => uri.startsWith(r.split('/callback')[0]));
214    if (!match) return html(res,
215      '<h2>invalid_request</h2><p>redirect_uri <code>' + esc(uri) + '</code> is not '

```

The authorization server and IdP

LINES 216–269 OF 591

```

216     + 'registered. This check is what stops an attacker having your authorization '
217     + 'code delivered to their own server.</p>', 400);
218     if (!policy.exact_redirect_uri)
219         log('warn', 'redirect_uri matched by PREFIX (unsafe)');
220
221     const state = q.get('state');
222
223     // Step 3: response_type. OAuth 2.1 means `code`, full stop.
224     if (q.get('response_type') !== 'code') return redirectError(res, uri,
225         'unsupported_response_type',
226         'OAuth 2.1 supports only response_type=code. The implicit grant was removed '
227         + 'because it delivered access tokens through the browser URL.', state);
228
229     // Step 4: PKCE. The defining requirement of OAuth 2.1.
230     const challenge = q.get('code_challenge');
231     const method = q.get('code_challenge_method');
232     if (!challenge) {
233         if (policy.require_pkce) return redirectError(res, uri, 'invalid_request',
234             'code_challenge is required. OAuth 2.1 mandates PKCE for every '
235             + 'authorization code request, for public and confidential clients alike.',
236             state);
237         log('warn', 'PROCEEDING WITHOUT PKCE - this code will be stealable');
238     } else {
239         if (!method) return redirectError(res, uri, 'invalid_request',
240             'Send code_challenge_method=S256 explicitly. If omitted, RFC 7636 '
241             + 'defaults to "plain", which offers no protection.', state);
242         if (method !== 'S256') return redirectError(res, uri, 'invalid_request',
243             'code_challenge_method must be S256. With "plain" the challenge IS the '
244             + 'verifier, so anyone who sees the request can redeem the code.', state);
245         if (!/^[A-Za-z0-9-._]{43}$/.test(challenge)) return redirectError(res, uri,
246             'invalid_request', 'An S256 code_challenge is 43 base64url characters. '
247             + 'Did you send the verifier by mistake?', state);
248         log('ok', 'PKCE challenge stored: ' + challenge.slice(0, 16) + '...');
249     }
250
251     // Step 5: scope, capped by what this client is registered for.
252     const wanted = splitScope(q.get('scope'));
253     if (!wanted.length) return redirectError(res, uri, 'invalid_scope',
254         'No scope requested.', state);
255     const bad = wanted.filter((s) => !client.scopes.includes(s));
256     if (bad.length) return redirectError(res, uri, 'invalid_scope',
257         'Client is not registered for: ' + bad.join(' '), state);
258
259     // Step 6: park the request and hand over to the IdP.
260     const rid = C.randomId(16);
261     requests.set(rid, { clientId: q.get('client_id'), uri, scope: wanted, state,
262         nonce: q.get('nonce'), challenge, method });
263     return resume(res, rid, cookies(req).lab_session);
264 }
265
266 // ----- IdP: authentication -----
267 if (path === '/login' && req.method === 'POST') {
268     const rid = body.get('request_id'), r = requests.get(rid);
269     if (!r) return html(res, '<h2>Expired</h2><p>Start again.</p>', 400);

```

The authorization server and IdP

LINES 270–323 OF 591

```

270     const user = USERS[String(body.get('username') || '').toLowerCase()];
271     // One generic message for both cases: telling them apart is a free
272     // user-enumeration oracle for an attacker.
273     if (!user || !C.safeEqual(body.get('password'), user.password)) {
274         log('reject', 'authentication failed');
275         return loginPage(res, rid, r, 'That username and password is not correct.');
```

```

276     }
```

```

277     const sid = C.randomToken(24);
```

```

278     sessions.set(sid, { sub: user.sub, authTime: now(),
```

```

279         expiresAt: now() + TTL.session });
```

```

280     log('ok', 'user authenticated: ' + user.name);
```

```

281     res.setHeader('set-cookie', 'lab_session=' + sid
```

```

282         + '; Path=/; HttpOnly; SameSite=Lax; Max-Age=' + TTL.session);
```

```

283     return resume(res, rid, sid);
```

```

284 }
```

```

285
```

```

286 // ----- OAuth: consent, then the code -----
```

```

287 if (path === '/consent' && req.method === 'POST') {
```

```

288     const rid = body.get('request_id'), r = requests.get(rid);
```

```

289     if (!r) return html(res, '<h2>Expired</h2>', 400);
```

```

290     const s = sessions.get(cookies(req).lab_session);
```

```

291     if (!s) return resume(res, rid, null);
```

```

292     const user = userBySub(s.sub);
```

```

293
```

```

294     if (body.get('action') !== 'allow') {
```

```

295         requests.delete(rid);
```

```

296         return redirectError(res, r.uri, 'access_denied',
```

```

297             'The resource owner refused. Every client must handle this.', r.state);
```

```

298     }
```

```

299     // Approved = asked for AND the user holds it AND the user ticked it.
```

```

300     const ticked = body.getAll('scope');
```

```

301     const granted = r.scope.filter((x) =>
```

```

302         user.can.includes(x) && (ticked.includes(x) || x === 'openid'));
```

```

303     if (!granted.length) {
```

```

304         requests.delete(rid);
```

```

305         return redirectError(res, r.uri, 'access_denied', 'No scopes granted.', r.state);
```

```

306     }
```

```

307     if (granted.length !== r.scope.length)
```

```

308         log('warn', 'granted scope is NARROWER than requested: ' + granted.join(' '));
```

```

309
```

```

310     // Bind to the code everything /token must re-check: the client, the
```

```

311     // redirect_uri, and the PKCE challenge. The code itself means nothing.
```

```

312     const code = C.randomToken(32);
```

```

313     codes.set(code, { clientId: r.clientId, sub: s.sub, scope: granted, uri: r.uri,
```

```

314         challenge: r.challenge, method: r.method, nonce: r.nonce,
```

```

315         authTime: s.authTime, used: false, expiresAt: now() + TTL.code });
```

```

316     requests.delete(rid);
```

```

317     log('ok', 'code issued: ' + code.slice(0, 12) + '... scope=' + granted.join(' '));
```

```

318
```

```

319     const out = new URL(r.uri);
```

```

320     out.searchParams.set('code', code);
```

```

321     if (r.state) out.searchParams.set('state', r.state);
```

```

322     out.searchParams.set('iss', ISSUER); // RFC 9207: mix-up defence
```

```

323     return redirect(res, out.toString());
```

The authorization server and IdP

LINES 324–377 OF 591

```

324   }
325
326   // ----- /token : the BACK CHANNEL -----
327   if (path === '/token' && req.method === 'POST') {
328     const grant = body.get('grant_type');
329     log('step', 'POST /token grant_type=' + grant);
330
331     if (grant === 'password' || grant === 'implicit') return json(res, 400, {
332       error: 'unsupported_grant_type',
333       error_description: 'Removed in OAuth 2.1. The password grant made the app '
334         + 'handle the real password: no consent, no MFA, no federation. Use '
335         + 'authorization_code + PKCE.' });
336
337     const auth = authenticateClient(req, body);
338     if (auth.error) {
339       res.setHeader('www-authenticate', 'Basic realm="lab"');
340       return json(res, 401,
341         { error: auth.error, error_description: auth.description });
342     }
343     const { id: clientId, client } = auth;
344     if (!client.grants.includes(grant)) return json(res, 400, {
345       error: 'unauthorized_client',
346       error_description: 'Client is not registered for grant_type=' + grant });
347
348     // ===== authorization_code =====
349     if (grant === 'authorization_code') {
350       const code = body.get('code');
351       const entry = codes.get(code);
352       if (!entry) return json(res, 400, { error: 'invalid_grant',
353         error_description: 'Unknown authorization code.' });
354
355       // Replay detection. A code is a one-shot credential (RFC 9700 4.1.3).
356       if (entry.used) {
357         if (policy.single_use_codes) {
358           const killed = entry.familyId ? revokeFamily(entry.familyId) : 0;
359           log('reject', 'CODE REPLAY - revoked ' + killed + ' token(s) it produced');
360           return json(res, 400, { error: 'invalid_grant',
361             error_description: 'This code was already used. A replay means someone '
362               + 'else may hold it, so every token derived from it is now revoked.' });
363         }
364         log('warn', 'CODE REPLAY ALLOWED - single_use_codes is off');
365       }
366       if (entry.expiresAt < now()) return json(res, 400, { error: 'invalid_grant',
367         error_description: 'Code expired. Codes live ~' + TTL.code + 's because they '
368           + 'cross the browser.' });
369       if (entry.clientId !== clientId) {
370         log('reject', 'code substitution blocked: issued to ' + entry.clientId);
371         return json(res, 400, { error: 'invalid_grant',
372           error_description: 'This code was issued to a different client.' });
373       }
374       if (entry.uri !== body.get('redirect_uri')) return json(res, 400, {
375         error: 'invalid_grant',
376         error_description: 'redirect_uri does not match the authorization request.' });
377

```

The authorization server and IdP

LINES 378–431 OF 591

```

378 // ===== the PKCE check: prove you started this flow =====
379 if (entry.challenge) {
380   const r = C.verifyPkce(body.get('code_verifier'),
381                         entry.challenge, entry.method);
382   if (!r.ok) {
383     entry.used = true; // burn it: something is wrong
384     log('reject', 'PKCE FAILED (' + r.reason + ') expected='
385         + entry.challenge.slice(0, 16) + '... got='
386         + String(r.computed).slice(0, 16));
387     return json(res, 400, { error: 'invalid_grant',
388       error_description: 'PKCE check failed (' + r.reason + '). The hash of the '
389       + 'presented code_verifier does not equal the stored code_challenge, so '
390       + 'whoever sent this did not start the flow.' });
391   }
392   log('ok', 'PKCE verified: SHA256(verifier) == stored challenge');
393 } else {
394   log('warn', 'no PKCE binding - anyone holding this code could redeem it');
395 }
396
397 entry.used = true;
398 const familyId = C.randomId(8);
399 entry.familyId = familyId;
400 const user = userBySub(entry.sub);
401 const at = issueAccessToken(clientId, entry.sub, entry.scope, familyId);
402 const out = { access_token: at.token, token_type: 'Bearer',
403             expires_in: TTL.access, scope: entry.scope.join(' ') };
404
405 if (client.grants.includes('refresh_token'))
406   out.refresh_token = issueRefreshToken(clientId, entry.sub, entry.scope,
407                                       familyId, 1);
408 if (entry.scope.includes('openid')) {
409   // An ID token is for the CLIENT: aud is the client_id, not the API.
410   const iat = now();
411   out.id_token = C.signJwt({ iss: ISSUER, sub: user.sub, aud: clientId,
412     iat, exp: iat + TTL.access, auth_time: entry.authTime,
413     nonce: entry.nonce || undefined,
414     name: entry.scope.includes('profile') ? user.name : undefined,
415     email: entry.scope.includes('email') ? user.email : undefined }, KEY, 'JWT');
416 }
417 log('ok', 'code exchanged for tokens');
418 return json(res, 200, out);
419 }
420
421 // ===== refresh_token =====
422 if (grant === 'refresh_token') {
423   const presented = body.get('refresh_token');
424   const hash = C.sha256hex(String(presented || ''));
425   const rec = refresh.get(hash);
426   if (!rec) return json(res, 400, { error: 'invalid_grant',
427     error_description: 'Unknown refresh token.' });
428   if (rec.clientId !== clientId) return json(res, 400, { error: 'invalid_grant',
429     error_description: 'This refresh token belongs to a different client.' });
430   if (rec.revoked) return json(res, 400, { error: 'invalid_grant',
431     error_description: 'Revoked, most likely because a replay was detected '

```

The authorization server and IdP

LINES 432–485 OF 591

```

432         + 'earlier in its family.' });
433
434     // ===== reuse detection: the whole point of rotation (RFC 9700 4.14) =====
435     if (rec.used) {
436         if (policy.detect_reuse) {
437             const killed = revokeFamily(rec.familyId);
438             log('reject', 'REFRESH REUSE - revoked family of ' + killed);
439             return json(res, 400, { error: 'invalid_grant',
440                 error_description: 'This refresh token was already used (generation '
441                     + rec.generation + '). Either it was stolen and replayed, or the real '
442                     + 'client lost the rotated value. The server cannot tell, so the whole '
443                     + 'family is revoked and the user must sign in again.' });
444         }
445         log('warn', 'reused refresh token ACCEPTED - detect_reuse is off');
446     }
447     if (rec.expiresAt < now()) return json(res, 400, { error: 'invalid_grant',
448         error_description: 'Refresh token expired.' });
449
450     // Scope may only ever shrink on refresh.
451     let scope = rec.scope;
452     if (body.get('scope')) {
453         const asked = splitScope(body.get('scope'));
454         const up = asked.filter((s) => !rec.scope.includes(s));
455         if (up.length) return json(res, 400, { error: 'invalid_scope',
456             error_description: 'Cannot widen scope on refresh: ' + up.join(' ') });
457         scope = asked;
458     }
459
460     rec.used = true;
461     const at = issueAccessToken(clientId, rec.sub, scope, rec.familyId);
462     const out = { access_token: at.token, token_type: 'Bearer',
463         expires_in: TTL.access, scope: scope.join(' ') };
464     if (policy.rotate_refresh) {
465         out.refresh_token = issueRefreshToken(clientId, rec.sub, scope, rec.familyId,
466             rec.generation + 1);
467         log('ok', 'refresh ROTATED to generation ' + (rec.generation + 1));
468     } else {
469         rec.used = false;
470         out.refresh_token = presented;
471         log('warn', 'refresh NOT rotated - the same value keeps being reused');
472     }
473     return json(res, 200, out);
474 }
475
476 // ===== client_credentials: no user, so nobody to consent =====
477 if (grant === 'client_credentials') {
478     const asked = body.get('scope') ? splitScope(body.get('scope')) : client.scopes;
479     const bad = asked.filter((s) => !client.scopes.includes(s));
480     if (bad.length) return json(res, 400, { error: 'invalid_scope',
481         error_description: 'Not registered for: ' + bad.join(' ') });
482     // sub is the CLIENT. Never use this grant to act on a user's behalf.
483     // `resource` (RFC 8707) lets the caller name which API the token is for;
484     // Lab 8 uses it to mint a token this API must refuse.
485     const at = issueAccessToken(clientId, clientId, asked, null,

```

The authorization server and IdP

LINES 486–538 OF 591

```

486         body.get('resource'));
487     log('ok', 'client_credentials token issued (no user involved)');
488     return json(res, 200, { access_token: at.token, token_type: 'Bearer',
489         expires_in: TTL.access, scope: asked.join(' ') });
490 }
491
492 return json(res, 400, { error: 'unsupported_grant_type' });
493 }
494
495 // ----- introspection (RFC 7662) -----
496 // Must be protected: it answers questions about other people's tokens.
497 if (path === '/introspect' && req.method === 'POST') {
498     const auth = authenticateClient(req, body);
499     if (auth.error || auth.client.type !== 'confidential')
500         return json(res, 401, { error: 'invalid_client' });
501     const crypto = require('node:crypto');
502     const r = C.verifyJwt(body.get('token'), {
503         getKey: (kid) => kid === KEY.kid
504             ? crypto.createPublicKey({ key: KEY.publicJwk, format: 'jwk' }) : null,
505         issuer: ISSUER });
506     if (!r.valid) return json(res, 200, { active: false, reason: r.reason });
507     if (revoked.has(r.payload.jti)) return json(res, 200, { active: false,
508         reason: 'revoked',
509         note: 'The signature is fine but this jti was revoked before exp. Only '
510             + 'introspection can see that – local JWT validation cannot.' });
511     return json(res, 200, { active: true, scope: r.payload.scope, sub: r.payload.sub,
512         client_id: r.payload.client_id, aud: r.payload.aud, exp: r.payload.exp,
513         jti: r.payload.jti, token_type: 'Bearer' });
514 }
515
516 // ----- revocation (RFC 7009) -----
517 // Always answers 200, even for an unknown token: this endpoint must not
518 // become an oracle for guessing valid ones.
519 if (path === '/revoke' && req.method === 'POST') {
520     const auth = authenticateClient(req, body);
521     if (auth.error) return json(res, 401, { error: auth.error });
522     const token = body.get('token');
523     const rec = refresh.get(C.sha256hex(String(token || '')));
524     if (rec) {
525         const n = revokeFamily(rec.familyId);
526         log('ok', 'revoked refresh family (' + n + ' tokens)');
527         return json(res, 200, { revoked: true, family_tokens: n });
528     }
529     const d = C.decodeJwt(token);
530     if (d && d.payload.jti) {
531         revoked.add(d.payload.jti);
532         log('ok', 'revoked access token jti=' + d.payload.jti);
533         return json(res, 200, { revoked: true,
534             note: 'An API doing local JWT validation will keep accepting this until exp. '
535                 + 'That is the trade-off of self-contained tokens.' });
536     }
537     return json(res, 200,
538         { revoked: true, note: 'Unknown token; 200 anyway per RFC 7009.' });

```

The authorization server and IdP

LINES 539–591 OF 591

```

539   }
540
541   return json(res, 404, { error: 'not_found', path });
542 });
543
544 // --- the IdP's two screens -----
545 function resume(res, rid, sid) {
546   const r = requests.get(rid);
547   if (!r) return html(res, '<h2>Expired</h2>', 400);
548   const s = sessions.get(sid);
549   if (!s || s.expiresAt < now()) return loginPage(res, rid, r);
550   return consentPage(res, rid, r, userBySub(s.sub));
551 }
552
553 function loginPage(res, rid, r, error) {
554   return html(res,
555     '<h2>Sign in</h2><p><b>' + esc(r.clientId) + '</b> wants to act on your behalf. '
556     + '<b>Notice you are typing your password into the <b>identity provider</b> '
557     + '(localhost:7000), never into the application. That is the whole point.</p>'
558     + (error ? '<p style="color:#c0203f"><b>' + esc(error) + '</b></p>' : '')
559     + '<form method=POST action=/login>'
560     + '<input type=hidden name=request_id value="' + esc(rid) + '">'
561     + '<p><label>Username<br><input name=username autofocus></label>'
562     + '<p><label>Password<br><input name=password type=password></label>'
563     + '<p><button>Sign in</button></form>'
564     + '<hr><p style="color:#666;font-size:13px">alice / wonderland &middot; '
565     + 'bob / builder (bob cannot grant payments:write)</p>');
566 }
567
568 function consentPage(res, rid, r, user) {
569   const grantable = r.scope.filter((s) => user.can.includes(s));
570   const refused = r.scope.filter((s) => !user.can.includes(s));
571   return html(res,
572     '<h2>Allow access?</h2><p>Signed in as <b>' + esc(user.name) + '</b>. '
573     + '<b>' + esc(r.clientId) + '</b> is asking for:</p>'
574     + (refused.length ? '<p style="color:#a8620a">' + esc(user.name) + ' cannot '
575       + 'delegate <code>' + esc(refused.join(' ')) + '</code>, so it is dropped. A '
576       + 'client never receives more authority than the user has.</p>' : '')
577     + '<form method=POST action=/consent>'
578     + '<input type=hidden name=request_id value="' + esc(rid) + '">'
579     + grantable.map((s) => '<p><label><input type=checkbox name=scope value="'
580       + esc(s) + '" checked' + (s === 'openid' ? ' disabled' : '') + '> <code>'
581       + esc(s) + '</code></label>').join('')
582     + '<p><button name=action value=allow>Allow</button> '
583     + '<button name=action value=deny>Deny</button></form>');
584 }
585
586 server.listen(PORT, () => {
587   console.log('\n Authorization server + IdP ' + ISSUER);
588   console.log(' discovery ' + ISSUER + '/.well-known/oauth-authorization-server');
589   console.log(' policy ' + ISSUER + '/policy (turn rules off for Lab 3)');
590   console.log(' signing kid ' + KEY.kid + '\n');
591 });

```

The resource server

The four gates of Chapter 11, in order, in one function. Note that steps 1–3 never contact the authorization server: the signature is checked offline against cached public keys.

LINES 1–49 OF 196

```

1  'use strict';
2  // =====
3  // LAB FILE 3 of 4 – api-server.js          run: node api-server.js
4  // The resource server: the API that owns the data. On every request it runs
5  // the four checks from Chapter 11, and nothing else.
6  // =====
7  const http = require('node:http');
8  const crypto = require('node:crypto');
9  const C = require('./crypto-lab');
10
11  const PORT = 7010;
12  const AUDIENCE = 'http://localhost:7010';      // tokens must name US in `aud`
13  const TRUSTED_ISSUER = 'http://localhost:7000'; // and be signed by THEM
14
15  // The API's own data. The token says WHO is asking and WHAT they may do; the
16  // data belongs to the API, not to the token.
17  const DB = {
18    'u-1001': { profile: { name: 'Alice Liddell', tier: 'gold' }, payments: [],
19              accounts: [{ id: 'ACC-4410', label: 'Everyday Checking', balance: 4820.55 },
20                        { id: 'ACC-9982', label: 'Rainy Day Savings', balance: 15200.0 } ] },
21    'u-1002': { profile: { name: 'Bob Builder', tier: 'standard' }, payments: [],
22              accounts: [{ id: 'ACC-7100', label: 'Joint Account', balance: 310.2 } ] },
23  };
24
25  // --- the JWKS cache -----
26  // Fetch the issuer's public keys once, cache them by kid, and refetch when an
27  // unknown kid appears. That is how key rotation works with no shared secret
28  // and no downtime. Restarting the AS rotates its key – watch this recover.
29  const jwks = { keys: new Map(), fetchedAt: 0 };
30
31  async function refreshJwks(why) {
32    const r = await fetch(TRUSTED_ISSUER + '/jwks.json');
33    const body = await r.json();
34    jwks.keys.clear();
35    for (const jwk of body.keys)
36      jwks.keys.set(jwk.kid, crypto.createPublicKey({ key: jwk, format: 'jwk' }));
37    jwks.fetchedAt = Date.now();
38    log('ok', 'JWKS fetched (' + why + '): ' + [...jwks.keys.keys()].join(', '));
39  }
40
41  async function keyFor(kid) {
42    if (!jwks.keys.size) await refreshJwks('cold start');
43    if (kid && !jwks.keys.has(kid)) await refreshJwks('unknown kid ' + kid);
44    return jwks.keys.get(kid) || null;
45  }
46
47  const log = (tag, msg) => console.log(' [api] ' + tag.padEnd(7) + msg);
48  const json = (res, status, obj) => (res.writeHead(status,
49    { 'content-type': 'application/json' }), res.end(JSON.stringify(obj, null, 2)));

```

The resource server

LINES 50-98 OF 196

```

50
51 // =====
52 // The four checks, in order. This function IS the resource server.
53 // =====
54 async function authorize(req, res, required) {
55   // 1. Is there a Bearer token?                                -> 401
56   const header = req.headers.authorization || '';
57   if (!header.toLowerCase().startsWith('bearer ')) {
58     res.setHeader('www-authenticate',
59       'Bearer realm="" + AUDIENCE + "", scope="" + required.join(' ') + ""');
60     log('401', 'no Bearer token');
61     return { body: { error: 'invalid_token',
62       error_description: 'This endpoint needs an OAuth 2.1 Bearer access token.' },
63       status: 401 };
64   }
65   const token = header.slice(7).trim();
66
67   // 2. Is the signature real and mine to trust?                -> 401
68   // 3. Was it minted for THIS api, and is it still live? -> 401
69   const decoded = C.decodeJwt(token);
70   const kid = decoded && decoded.header ? decoded.header.kid : null;
71   const key = await keyFor(kid);
72   let result = C.verifyJwt(token, { getKey: () => key, issuer: TRUSTED_ISSUER,
73     audience: AUDIENCE, requiredTyp: 'at+jwt' });
74
75   // A signature failure on a kid we thought we knew usually means the key
76   // behind it changed. Refetch once before blaming the caller.
77   if (!result.valid && result.reason === 'bad_signature'
78     && Date.now() - jwks.fetchedAt > 5000) {
79     await refreshJwks('signature failure on a known kid');
80     result = C.verifyJwt(token, { getKey: () => jwks.keys.get(kid) || null,
81       issuer: TRUSTED_ISSUER, audience: AUDIENCE, requiredTyp: 'at+jwt' });
82   }
83
84   if (!result.valid) {
85     res.setHeader('www-authenticate', 'Bearer error="invalid_token"');
86     log('401', result.reason);
87     return { status: 401, body: { error: 'invalid_token', check_failed: result.reason,
88       error_description: {
89         unexpected_typ: 'Expected typ="at+jwt". You almost certainly sent the ID '
90           + 'TOKEN instead of the access token - a very common bug.',
91         audience_mismatch: 'This token was minted for another audience. A perfect '
92           + 'signature does not make a token valid HERE.',
93         unsupported_alg: 'Only RS256 is accepted. Never trust the token\'s own alg.',
94         unknown_kid: 'No published key matches. The AS may have rotated keys.',
95         bad_signature: 'Altered, or signed by something other than the issuer.',
96         token_expired: 'Access tokens are short-lived on purpose. Refresh it.',
97         issuer_mismatch: 'This API only trusts ' + TRUSTED_ISSUER + '.',
98       }[result.reason] || result.reason } } };

```

The resource server

LINES 99–147 OF 196

```

99   }
100
101   // 4. Does the scope cover THIS operation?           -> 403, not 401
102   const granted = String(result.payload.scope || '').split(/\s+/).filter(Boolean);
103   const missing = required.filter((s) => !granted.includes(s));
104   if (missing.length) {
105     res.setHeader('www-authenticate', 'Bearer error="insufficient_scope", scope="'
106       + required.join(' ') + '"');
107     log('403', 'insufficient_scope, needs ' + missing.join(' '));
108     return { status: 403, body: { error: 'insufficient_scope',
109       required_scope: required, granted_scope: granted,
110       error_description: 'The token is perfectly valid – it simply does not '
111         + 'authorize this action. That is why this is 403 and not 401: refreshing '
112         + 'will not help, the user has to grant more.' } }};
113   }
114
115   log('ok', req.method + ' ' + req.url + ' sub=' + result.payload.sub
116     + ' scope="' + result.payload.scope + '"');
117   return { claims: result.payload };
118 }
119
120 // =====
121 const server = http.createServer(async (req, res) => {
122   const url = new URL(req.url, AUDIENCE);
123   const path = url.pathname;
124
125   if (path === '/healthz') return json(res, 200,
126     { ok: true, audience: AUDIENCE, cached_kids: [...jwks.keys.keys()] });
127
128   // ---- profile, needs `profile` ----
129   if (path === '/api/me') {
130     const a = await authorize(req, res, ['profile']);
131     if (a.status) return json(res, a.status, a.body);
132     const rec = DB[a.claims.sub];
133     // A client_credentials token has a client_id as its sub, and no human
134     // behind it – so there is no profile to return.
135     if (!rec) return json(res, 404, { error: 'no_such_subject', sub: a.claims.sub,
136       note: 'Valid token, but sub is not a user. This is what a machine token '
137         + 'looks like here.' });
138     return json(res, 200, { sub: a.claims.sub, ...rec.profile });
139   }
140
141   // ---- accounts, needs `accounts:read` ----
142   if (path === '/api/accounts') {
143     const a = await authorize(req, res, ['accounts:read']);
144     if (a.status) return json(res, a.status, a.body);
145     const rec = DB[a.claims.sub];
146     return json(res, 200, { sub: a.claims.sub, accounts: rec ? rec.accounts : [],
147       note: 'The API picked the data from the token\'s `sub`, never from a user id '

```

The resource server

LINES 148–196 OF 196

```

148     + 'in the request. Trusting a caller-supplied id here is how you build an '
149     + 'IDOR on top of perfectly good OAuth.' });
150   }
151
152   // ---- payments, needs `payments:write` ----
153   if (path === '/api/payments' && req.method === 'POST') {
154     const a = await authorize(req, res, ['payments:write']);
155     if (a.status) return json(res, a.status, a.body);
156     const rec = DB[a.claims.sub];
157     if (!rec) return json(res, 404, { error: 'no_such_subject' });
158     const payment = { id: 'PAY-' + Math.random().toString(36).slice(2, 8).toUpperCase(),
159       from: rec.accounts[0].id, to: 'ACC-EXTERNAL', amount: 42, status: 'accepted' };
160     rec.payments.push(payment);
161     return json(res, 201, { payment });
162   }
163
164   // ---- the same data, authorized by introspection instead (Lab 8) ----
165   // Slower - a network call per request - but it sees revocation instantly.
166   if (path === '/api/accounts-introspected') {
167     const header = req.headers.authorization || '';
168     const token = header.toLowerCase().startsWith('bearer ') ? header.slice(7) : '';
169     const r = await fetch(TRUSTED_ISSUER + '/introspect', { method: 'POST',
170       headers: { 'content-type': 'application/x-www-form-urlencoded',
171         authorization: 'Basic '
172           + Buffer.from('demo-service:service-secret').toString('base64') },
173       body: new URLSearchParams({ token }).toString() });
174     const intro = await r.json();
175     if (!intro.active) {
176       log('401', 'introspection says not active: ' + intro.reason);
177       return json(res, 401, { error: 'invalid_token', introspection: intro,
178         note: 'Local JWT validation would still have accepted a revoked-but-'
179           + 'unexpired token. Introspection is the only way an API learns about '
180           + 'revocation immediately - at the cost of a round trip per call.' });
181     }
182     const rec = DB[intro.sub];
183     log('ok', 'introspection says active, sub=' + intro.sub);
184     return json(res, 200, { sub: intro.sub, accounts: rec ? rec.accounts : [],
185       validated_by: 'token introspection (RFC 7662)' });
186   }
187
188   return json(res, 404, { error: 'not_found', path });
189 });
190
191 server.listen(PORT, () => {
192   console.log('\n Resource server ' + AUDIENCE);
193   console.log(' trusts issuer ' + TRUSTED_ISSUER);
194   console.log(' scopes /api/me=profile /api/accounts=accounts:read'
195     + ' /api/payments=payments:write\n');
196 });

```

The client, and all ten labs

The client walks the front channel with a cookie jar instead of a browser, so every lab is one command whose result is printed rather than clicked. Each LABS.* entry is one exercise.

LINES 1–53 OF 450

```

1  'use strict';
2  // =====
3  // LAB FILE 4 of 4 – client.js           run: node client.js <lab>
4  // The OAuth 2.1 client, plus every lab scenario. It walks the front channel
5  // with a cookie jar instead of a browser, so each lab is one command and the
6  // result is printed, not clicked.
7  //
8  //   node client.js list           show every lab
9  //   node client.js happy         Lab 1 the correct flow
10 //   node client.js all           run all of them
11 // =====
12 const C = require('./crypto-lab');
13
14 const AS = 'http://localhost:7000';
15 const API = 'http://localhost:7010';
16 const REDIRECT = 'http://localhost:7020/callback';
17 const CLIENT = { id: 'demo-web-app', secret: 'web-app-secret' };
18 const SPA = { id: 'demo-spa', secret: null };
19 const SERVICE = { id: 'demo-service', secret: 'service-secret' };
20
21 // --- printing -----
22 const B = '\x1b[1m', D = '\x1b[2m', R = '\x1b[0m';
23 const CY = '\x1b[36m', GR = '\x1b[32m', RE = '\x1b[31m', YE = '\x1b[33m';
24 const step = (n, s) => console.log('\n' + CY + B + 'STEP ' + n + ' ' + s + R);
25 const why = (s) => console.log(' ' + D + s + R);
26 const val = (k, v) => console.log(' ' + k.padEnd(20) + ' ' + v);
27 const pass = (s) => console.log(' ' + GR + B + 'PASS' + R + ' ' + s);
28 const fail = (s) => console.log(' ' + RE + B + 'FAIL' + R + ' ' + s);
29 const note = (s) => console.log(' ' + YE + B + 'NOTE' + R + ' ' + s);
30 const head = (s) => console.log('\n' + B + '='.repeat(70) + '\n' + s + '\n'
31   + '='.repeat(70) + R);
32
33 // --- a browser, more or less -----
34 // Keeps one cookie and never follows redirects, so we can read the Location
35 // header the way the browser's address bar would show it.
36 function makeBrowser() {
37   let cookie = null;
38   return async function go(url, options = {}) {
39     const headers = { ...(options.headers || {}) };
40     if (cookie) headers.cookie = cookie;
41     const res = await fetch(url, { ...options, headers, redirect: 'manual' });
42     const set = res.headers.get('set-cookie');
43     if (set) cookie = set.split(';')[0];
44     return { status: res.status, location: res.headers.get('location'),
45       body: await res.text() };
46   };
47 }
48
49 const form = (obj) => ({ method: 'POST', body: new URLSearchParams(obj).toString(),
50   headers: { 'content-type': 'application/x-www-form-urlencoded' } });
51
52 // --- the token endpoint -----
53 async function token(params, client) {

```

The client, and all ten labs

LINES 54–110 OF 450

```

54   const headers = { 'content-type': 'application/x-www-form-urlencoded' };
55   const body = new URLSearchParams(params);
56   if (client.secret) {
57     // A confidential client authenticates. RFC 6749 2.3.1 wants the id and
58     // secret form-urlencoded before base64.
59     headers.authorization = 'Basic ' + Buffer.from(
60       encodeURIComponent(client.id) + ':' + encodeURIComponent(client.secret)
61     ).toString('base64');
62   } else {
63     body.set('client_id', client.id); // public: identified, not authenticated
64   }
65   const res = await fetch(AS + '/token', { method: 'POST', headers,
66     body: body.toString() });
67   return { status: res.status, body: await res.json() };
68 }
69
70 async function callApi(path, accessToken, method = 'GET') {
71   const headers = {};
72   if (accessToken) headers.authorization = 'Bearer ' + accessToken;
73   const res = await fetch(API + path, { method, headers });
74   return { status: res.status, body: await res.json() };
75 }
76
77 const setPolicy = (key, value) =>
78   fetch(AS + '/policy', form({ key, value: String(value) })).then((r) => r.json());
79
80 // =====
81 // The front channel, walked end to end. Returns the authorization code.
82 // =====
83 async function getCode({ client = CLIENT, scope = 'openid profile accounts:read',
84   user = 'alice', password = 'wonderland',
85   pkce = true, redirect = REDIRECT, quiet = false } = {}) {
86   const browser = makeBrowser();
87   const verifier = C.createVerifier();
88   const challenge = C.challengeS256(verifier);
89   const state = C.randomToken(16);
90
91   const q = new URLSearchParams({ response_type: 'code', client_id: client.id,
92     redirect_uri: redirect, scope, state, nonce: C.randomToken(12) });
93   if (pkce) { q.set('code_challenge', challenge);
94     q.set('code_challenge_method', 'S256'); }
95
96   if (!quiet) {
97     val('code_verifier', verifier);
98     why('43 random chars. Stays here; never crosses the browser.');
```

The client, and all ten labs

LINES 111–167 OF 450

```

111   if (!rid) return { error: 'no_login_form', description: r.body.slice(0, 200) };
112
113   // The password is typed into the IdP's own origin, not into this app.
114   r = await browser(AS + '/login', form({ request_id: rid, username: user, password }));
115   if (r.status === 302) r = await browser(AS + r.location.replace(AS, ''));
116   const offered = [...r.body.matchAll(/name=scope value="([^\"]+)/g)].map((m) => m[1]);
117
118   // Approve everything the consent screen actually offered.
119   const consent = new URLSearchParams({ request_id: rid, action: 'allow' });
120   for (const s of offered) consent.append('scope', s);
121   r = await browser(AS + '/consent', { method: 'POST', body: consent.toString(),
122     headers: { 'content-type': 'application/x-www-form-urlencoded' } });
123
124   const u = new URL(r.location);
125   if (u.searchParams.get('error'))
126     return { error: u.searchParams.get('error'),
127       description: u.searchParams.get('error_description'), verifier };
128   return { code: u.searchParams.get('code'), state: u.searchParams.get('state'),
129     iss: u.searchParams.get('iss'), verifier, challenge, sentState: state,
130     offered };
131 }
132
133 // =====
134 // The labs
135 // =====
136 const LABS = {};
137
138 // ----- Lab 1
139 LABS.happy = { title: 'The correct flow, end to end', run: async () => {
140   step(1, 'Create the PKCE pair, then walk the front channel');
141   const f = await getCode();
142   if (f.error) return fail(f.error + ': ' + f.description);
143   val('code', f.code);
144   val('state echoed', f.state);
145   val('iss (RFC 9207)', f.iss);
146   f.state === f.sentState ? pass('state matches – this callback is ours')
147     : fail('state mismatch');
148
149   step(2, 'Back channel: POST /token with the code AND the verifier');
150   why('The server re-hashes our verifier and compares it to the challenge it');
151   why('stored in step 1. Only the app that made the verifier can pass.');
```

The client, and all ten labs

LINES 168–224 OF 450

```

168     a.status === 200 ? pass('200 - ' + a.body.accounts.length + ' accounts')
169       : fail(JSON.stringify(a.body));
170
171     step(5, 'Ask for something the token is not scoped for');
172     const p = await callApi('/api/payments', t.body.access_token, 'POST');
173     p.status === 403 ? pass('403 insufficient_scope - the API enforced scope, not '
174       + 'just a valid signature')
175       : fail('expected 403, got ' + p.status);
176     why(p.body.error_description || '');
177     return t.body;
178   } };
179
180   // ----- Lab 2
181   LABS.pkce = { title: 'The PKCE transform, step by step', run: async () => {
182     const v = C.createVerifier();
183     const e = C.explainPkce(v);
184     step(1, 'One random string, one hash');
185     val('code_verifier', e.code_verifier);
186     val('length', e.length + ' characters');
187     step(2, 'SHA-256 of the verifier TEXT');
188     val('sha256 (hex)', e.sha256_hex);
189     val('sha256 (base64)', e.sha256_base64);
190     val('code_challenge', e.code_challenge);
191     why('base64url is base64 with + -> -, / -> _, and the = padding removed.');
```

step(3, 'What the server does with them');

```

192     pass('correct verifier -> ' + C.verifyPkce(v, e.code_challenge, 'S256').ok);
193     const wrong = C.verifyPkce(C.createVerifier(), e.code_challenge, 'S256');
194     pass('a different one -> ' + wrong.ok + ' (' + wrong.reason + ')');
195     why('The challenge is a hash, so publishing it gives an attacker nothing:');
196     why('reversing it means computing a SHA-256 preimage.');
```

197 } };

198

199

200 // ----- Lab 3

```

201 LABS.attack = { title: 'A stolen authorization code, refused by PKCE',
202   run: async () => {
203     step(1, 'A perfectly normal flow - and the code leaks');
204     const f = await getCode({ client: SPA, scope: 'openid accounts:read',
205       redirect: 'http://localhost:7020/spa-callback' });
206     if (f.error) return fail(f.error + ': ' + f.description);
207     val('stolen code', f.code);
208     why('Assume the worst: a malicious app, a leaked log, a crafted redirect.');
```

209 why('This is the PUBLIC client, so there is no secret to stop the attacker.');

210

```

211     step(2, 'The attacker redeems it - but has no code_verifier');
212     const a1 = await token({ grant_type: 'authorization_code', code: f.code,
213       redirect_uri: 'http://localhost:7020/spa-callback' }, SPA);
214     a1.status !== 200 ? pass('REFUSED: ' + a1.body.error)
215       : fail('the attack SUCCEEDED - is require_pkce off?');
216     why(a1.body.error_description || '');
217
218     step(3, 'The attacker guesses a verifier');
```

```

219     const a2 = await token({ grant_type: 'authorization_code', code: f.code,
220       redirect_uri: 'http://localhost:7020/spa-callback',
221       code_verifier: C.createVerifier() }, SPA);
222     a2.status !== 200 ? pass('REFUSED: the hash of a guess never matches')
223       : fail('unexpected success');
```

224 note('The verifier is 256 bits of randomness. Guessing it is not an attack,');

The client, and all ten labs

LINES 225–281 OF 450

```

225   note('it is a lottery with no winning ticket. THAT is what PKCE buys you.');
```

```

226 } };
```

```

227
```

```

228 // ----- Lab 4
```

```

229 LABS.nopkce = { title: 'The same theft with PKCE turned OFF', run: async () => {
```

```

230   note('Turning require_pkce off. This is the pre-2013 world.');
```

```

231   await setPolicy('require_pkce', false);
```

```

232   step(1, 'Get a code with no code_challenge at all');
```

```

233   const f = await getCode({ client: SPA, scope: 'openid accounts:read',
```

```

234     redirect: 'http://localhost:7020/spa-callback', pkce: false });
```

```

235   if (f.error) { await setPolicy('require_pkce', true);
```

```

236     return fail(f.error + ': ' + f.description); }
```

```

237   val('stolen code', f.code);
```

```

238
```

```

239   step(2, 'The attacker redeems it with nothing but the code');
```

```

240   const a = await token({ grant_type: 'authorization_code', code: f.code,
```

```

241     redirect_uri: 'http://localhost:7020/spa-callback' }, SPA);
```

```

242   if (a.status === 200) {
```

```

243     fail('ATTACK SUCCEEDED - the attacker now holds a real access token');
```

```

244     val('access_token', a.body.access_token.slice(0, 40) + '...');
```

```

245     const d = await callApi('/api/accounts', a.body.access_token);
```

```

246     if (d.status === 200) fail('and it WORKS: ' + JSON.stringify(d.body.accounts[0]));
```

```

247     note('The client_id is public, so the code alone was enough. This is the');
```

```

248     note('exact vulnerability OAuth 2.1 closes by making PKCE mandatory.');
```

```

249   } else { pass('refused: ' + a.body.error); }
```

```

250
```

```

251   step(3, 'Restore the safe default');
```

```

252   await setPolicy('require_pkce', true);
```

```

253   pass('require_pkce is back on');
```

```

254 } };
```

```

255
```

```

256 // ----- Lab 5
```

```

257 LABS.replay = { title: 'Replaying an already-used code', run: async () => {
```

```

258   const f = await getCode({ quiet: true });
```

```

259   step(1, 'First redemption (legitimate)');
```

```

260   const a = await token({ grant_type: 'authorization_code', code: f.code,
```

```

261     redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);
```

```

262   a.status === 200 ? pass('tokens issued') : fail(JSON.stringify(a.body));
```

```

263   step(2, 'Second redemption of the same code, same verifier');
```

```

264   const b = await token({ grant_type: 'authorization_code', code: f.code,
```

```

265     redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);
```

```

266   b.status !== 200 ? pass('REFUSED: codes are single-use')
```

```

267     : fail('replay accepted');
```

```

268   why(b.body.error_description || '');
```

```

269 } };
```

```

270
```

```

271 // ----- Lab 6
```

```

272 LABS.refresh = { title: 'Refresh rotation and reuse detection', run: async () => {
```

```

273   const f = await getCode({ quiet: true });
```

```

274   const first = await token({ grant_type: 'authorization_code', code: f.code,
```

```

275     redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);
```

```

276   const rt1 = first.body.refresh_token;
```

```

277   val('refresh gen 1', rt1.slice(0, 24) + '...');
```

```

278
```

```

279   step(1, 'Refresh once');
```

```

280   const second = await token({ grant_type: 'refresh_token', refresh_token: rt1 },
```

```

281     CLIENT);
```

The client, and all ten labs

LINES 282–338 OF 450

```

282     const rt2 = second.body.refresh_token;
283     rt2 && rt2 !== rt1 ? pass('ROTATED – a brand-new refresh token came back')
284         : fail('no rotation');
285     val('refresh gen 2', String(rt2).slice(0, 24) + '...');
286     why('The old value is now retired. Each refresh token is single-use.');
```

287

```

288     step(2, 'Replay the RETIRED generation 1, as a thief would');
289     const replay = await token({ grant_type: 'refresh_token', refresh_token: rt1 },
290         CLIENT);
291     replay.status !== 200 ? pass('REFUSED and the whole family revoked')
292         : fail('reuse accepted');
293     why(replay.body.error_description || '');
294
```

295

```

295     step(3, 'The legitimate generation 2 is collateral damage');
296     const after = await token({ grant_type: 'refresh_token', refresh_token: rt2 },
297         CLIENT);
298     after.status !== 200 ? pass('also refused – the server cannot tell thief from '
299         + 'victim, so it trusts neither')
300         : fail('gen 2 still works');
301     note('The user signs in again. That is the correct, safe outcome: a silent');
302     note('permanent compromise became a visible, recoverable one.');
```

303 } };

304

305 // ----- Lab 7

```

306 LABS.scope = { title: 'Scope is capped three times over', run: async () => {
307     step(1, 'Sign in as bob and ask for payments:write');
308     why('bob is not entitled to payments:write, so consent cannot offer it.');
```

309

```

309     const f = await getCode({ user: 'bob', password: 'builder',
310         scope: 'openid profile accounts:read payments:write', quiet: true });
311     if (f.error) return fail(f.error + ': ' + f.description);
312     val('requested', 'openid profile accounts:read payments:write');
313     val('offered at consent', f.offered.join(' '));
314     const t = await token({ grant_type: 'authorization_code', code: f.code,
315         redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);
316     val('actually granted', t.body.scope);
317     !t.body.scope.includes('payments:write')
318         ? pass('payments:write was dropped – the user never had it to give')
319         : fail('scope escalation!');
```

320

```

321     step(2, 'Use what bob DOES have');
322     const a = await callApi('/api/accounts', t.body.access_token);
323     a.status === 200 ? pass('200 on /api/accounts') : fail(JSON.stringify(a.body));
324
```

325

```

325     step(3, 'Now try to move money');
326     const p = await callApi('/api/payments', t.body.access_token, 'POST');
327     p.status === 403 ? pass('403 insufficient_scope') : fail('expected 403');
328     why('A valid token without the scope is 403, never 401. Refreshing will not');
329     why('help – the user has to grant more.');
```

330 } };

331

332 // ----- Lab 8

```

333 LABS.idtoken = { title: 'An ID token is not an access token', run: async () => {
334     const f = await getCode({ quiet: true });
335     const t = await token({ grant_type: 'authorization_code', code: f.code,
336         redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);
337
```

338

```

338     step(1, 'Compare the two tokens this one flow returned');
```

The client, and all ten labs

LINES 339–394 OF 450

```

339   const at = C.decodeJwt(t.body.access_token), it = C.decodeJwt(t.body.id_token);
340   val('access typ / aud', at.header.typ + ' / ' + at.payload.aud);
341   val('id typ / aud', it.header.typ + ' / ' + it.payload.aud);
342   why('The access token names the API. The ID token names the CLIENT.');
```

343

```

344   step(2, 'Send the ID token to the API as a Bearer token');
345   const bad = await callApi('/api/accounts', t.body.id_token);
346   bad.status === 401 ? pass('401 - refused, as it must be') : fail('ACCEPTED!');
347   val('check_failed', bad.body.check_failed);
348   why(bad.body.error_description || '');
349
```

```

350   step(3, 'Now a token minted for a DIFFERENT API (RFC 8707 `resource`)');
351   const m2m = await token({ grant_type: 'client_credentials',
352     scope: 'accounts:read', resource: 'http://localhost:9999' }, SERVICE);
353   const foreign = C.decodeJwt(m2m.body.access_token);
354   val('its aud', foreign.payload.aud);
355   why('Genuine signature, not expired, real issuer. The only thing wrong is aud.');
```

```

356   const rej = await callApi('/api/accounts', m2m.body.access_token);
357   rej.status === 401 ? pass('401 ' + rej.body.check_failed + ' - refused')
358     : fail('accepted a token for another API!');
359   why(rej.body.error_description || '');
360   note('Without the aud check, any API trusting this issuer would accept any');
361   note('token it ever minted. The audience is what confines a token to one API.');
```

362 } };

363

364 // ----- Lab 9

```

365 LABS.m2m = { title: 'Machine to machine: no user, no consent', run: async () => {
366   step(1, 'client_credentials');
367   const t = await token({ grant_type: 'client_credentials',
368     scope: 'accounts:read profile' }, SERVICE);
369   t.status === 200 ? pass('token issued with no browser and no consent screen')
370     : fail(JSON.stringify(t.body));
371   const d = C.decodeJwt(t.body.access_token);
372   val('sub', d.payload.sub);
373   val('refresh_token', t.body.refresh_token ? 'present' : 'none (correct)');
374   why('sub is the client_id: there is no human here. No refresh token either,');
375   why('because the client can simply ask again with its own credentials.');
```

376

```

377   step(2, 'Ask the API who the user is');
378   const me = await callApi('/api/me', t.body.access_token);
379   me.status === 404 ? pass('404 no_such_subject - correct, nobody delegated')
380     : note('status ' + me.status);
381   note('Never use this grant to act on a user\'s behalf. Nobody consented.');
```

382 } };

383

384 // ----- Lab 10

```

385 LABS.revoke = { title: 'Revocation: local validation vs introspection',
386   run: async () => {
387     const f = await getCode({ scope: 'openid accounts:read', quiet: true });
388     const t = await token({ grant_type: 'authorization_code', code: f.code,
389       redirect_uri: REDIRECT, code_verifier: f.verifier }, CLIENT);
390     const at = t.body.access_token;
```

391

```

392     step(1, 'Both endpoints work while the token is live');
393     val('local validation', (await callApi('/api/accounts', at)).status);
394     val('introspection', (await callApi('/api/accounts-introspected', at)).status);
```

The client, and all ten labs

LINES 395–450 OF 450

```

395
396   step(2, 'Revoke the access token (RFC 7009)');
397   const r = await fetch(AS + '/revoke', { method: 'POST',
398     headers: { 'content-type': 'application/x-www-form-urlencoded',
399       authorization: 'Basic ' + Buffer.from(CLIENT.id + ':' + CLIENT.secret)
400         .toString('base64') },
401     body: new URLSearchParams({ token: at }).toString() });
402   pass('revoked: ' + JSON.stringify(await r.json()));
403
404   step(3, 'Call both endpoints again');
405   const local = await callApi('/api/accounts', at);
406   const intro = await callApi('/api/accounts-introspected', at);
407   val('local validation', local.status + (local.status === 200
408     ? ' <- STILL ACCEPTED' : ''));
409   val('introspection', intro.status + (intro.status === 401
410     ? ' <- correctly refused' : ''));
411   note('A self-contained JWT knows nothing about being revoked: its signature');
412   note('and exp are unchanged. Introspection asks the server and sees the');
413   note('truth, at the cost of a round trip per call. THAT is why access tokens');
414   note('are given lifetimes measured in minutes.');
```

```

415 } };
416
417 // =====
418 const ORDER = ['happy', 'pkce', 'attack', 'nopkce', 'replay', 'refresh',
419   'scope', 'idtoken', 'm2m', 'revoke'];
420
421 async function main() {
422   const which = process.argv[2] || 'list';
423
424   if (which === 'list') {
425     console.log('\n' + B + ' OAuth 2.1 + PKCE labs' + R + '\n');
426     ORDER.forEach((k, i) => console.log(' ' + String(i + 1).padStart(2)
427       + ' ' + k.padEnd(10) + LABS[k].title));
428     console.log('\n node client.js <name> | node client.js all\n');
429     return;
430   }
431
432   // Both servers must be up before any lab can run.
433   for (const [name, url] of [['authorization server', AS], ['api', API]]) {
434     try { await fetch(url + '/healthz'); }
435     catch { console.log(RE + '\n Cannot reach the ' + name + ' at ' + url
436       + '\n Start it first: node ' + (url === AS ? 'authz-server'
437         : 'api-server') + '.js\n' + R); process.exit(1); }
438   }
439
440   const names = which === 'all' ? ORDER : [which];
441   for (const n of names) {
442     if (!LABS[n]) { console.log('Unknown lab: ' + n); process.exit(2); }
443     head('LAB ' + (ORDER.indexOf(n) + 1) + ' ' + LABS[n].title);
444     await LABS[n].run();
445   }
446   console.log('\n' + D + ' Watch the server side of all of this in the terminal'
447     + ' running authz-server.js\n' + R);
448 }
449
450 main().catch((e) => { console.error(e); process.exit(1); });

```

Glossary

Every term used in this guide, in one place.

access token

A credential saying what an application may do, on whose behalf, at which API, until when. Usually a signed JWT. Sent to the resource server.

authorization code

A short-lived, single-use reference handed to the client through the browser, exchanged at the token endpoint for tokens. Worthless on its own.

authorization server (AS)

The service that authenticates the user, collects consent, and issues tokens. Publishes its metadata and public keys.

aud — the “audience” claim: which recipient a token is for. A resource server must reject tokens naming anyone else.

back channel

A direct server-to-server HTTPS call, invisible to the browser. Safe for secrets. See Chapter 3.

bearer token

A token usable by whoever holds it, with no further proof required — like a password. Hence short lifetimes.

client

The application requesting access. **Confidential** if it can keep a secret (server-side), **public** if it cannot (SPA, mobile, CLI).

code challenge / code verifier

The PKCE pair. The verifier is a random secret held by the client; the challenge is its SHA-256 hash, safe to send through the browser.

front channel

Communication via browser redirects. Visible in URL bars, history, logs and `Referer` headers. Never carries secrets.

grant type

How a client obtains a token. OAuth 2.1 keeps `authorization_code`, `refresh_token` and `client_credentials`; `implicit` and `password` are gone.

identity provider (IdP)

The service holding user accounts and performing authentication. Usually packaged with the authorization server, but a distinct job.

ID token

An OpenID Connect JWT stating who signed in and when, addressed to the client. Not an API credential.

introspection — RFC 7662: an endpoint where a resource server can ask whether a token is still active. Sees revocation immediately.

iss — the “issuer” claim: which authorization server minted a token. Checked on every validation.

JWKS — JSON Web Key Set: the public keys an authorization server publishes so anyone can verify its signatures offline.

JWT

JSON Web Token: header, payload and signature, base64url-encoded and dot-separated. Signed, **not** encrypted.

kid — key id: names which published key signed a token, so keys can be rotated without downtime.

nonce

A random value echoed inside the ID token, proving it was minted for this login.

PKCE

Proof Key for Code Exchange (RFC 7636). Binds an authorization code to the application that requested it. Mandatory in OAuth 2.1.

refresh token

Exchanged for new access tokens without user interaction, and rotated on every use.

resource server

The API holding the data. Validates tokens and enforces scope.

scope

A label for a slice of authority (`accounts:read`): requested, capped by registration and entitlement, then approved by the user.

state

A random per-request value echoed back on the callback, so a client can recognise a flow it started.

The specifications

Which document says what, and a sane order to read them in.

Read these, in this order

DOCUMENT	WHAT IT GIVES YOU
RFC 9700 OAuth 2.0 Security BCP	Start here. The accumulated lessons of a decade of deployments, with the attacks explained. Almost everything OAuth 2.1 requires is justified in this document.
OAuth 2.1 draft-ietf-oauth-v2-1	The consolidated core: one specification with the dangerous options already removed. Read it after 9700 and it will feel inevitable.
RFC 7636 PKCE	Short and readable. The definition of <code>code_verifier</code> , <code>code_challenge</code> and <code>S256</code> , and the attack that motivated them.
RFC 6749 OAuth 2.0 core	The original framework. Still the normative text for most parameter definitions — but read it knowing which parts 2.1 deletes.
RFC 6750 Bearer token usage	How to send a token and what each error code means. This is where <code>401</code> vs <code>403 insufficient_scope</code> is defined.

The ones you will need eventually

DOCUMENT	WHAT IT COVERS
RFC 8414	Authorization server metadata: <code>/.well-known/oauth-authorization-server</code> , so clients discover endpoints instead of hard-coding them.
RFC 9068	JWT access tokens: the <code>at+jwt</code> type and the claims an API should expect.
RFC 7662 / 7009	Token introspection and revocation.
RFC 8252	OAuth for native apps: system browsers, custom URI schemes, and why an embedded web view is the wrong answer.
RFC 8707 / 9207	Resource indicators (audience selection) and the <code>iss</code> authorization-response parameter.
RFC 9449	DPoP: sender-constrained tokens, so a stolen token is useless without the client's key. The direction bearer tokens are heading.
OpenID Connect Core	Authentication on top of OAuth: ID tokens, <code>nonce</code> , <code>/userinfo</code> and the <code>openid</code> scope.

One last piece of advice. Do not write your own OAuth client or server for production — use a maintained library and a real authorization server. The value of understanding the protocol is in configuring it correctly, reading the error you actually got, and noticing when somebody proposes quietly removing one of these rules.

This guide accompanies the OAuth 2.1 + PKCE Simulator in `oauth-simulator/`. Regenerate it with `make guide` from `dev-guide/src/dev-guide.html`.